

EEE 475/575 Medical Image Reconstruction & Processing

Homework #3 Report

Özgür Ekin Sönmez
ID: 22201966

15 April 2026

GenAI Usage Disclosure

This homework was completed with the assistance of AntigraVity (Claude Opus 4.6 Thinking) by Google DeepMind/Anthropic. The tool was used to generate the MATLAB code structure and cell organization for all parts, especially creating plotting functions. Help to organize this report, including the mathematical derivations (Latex) in Part 2.5.

All submitted code and written content have been reviewed, understood, and verified by the student. The student is fully responsible for correctness and can explain and justify all results.

1 Part I – System Matrix

1.1 Part 1.1 – Viewing Calibration Data

The file `SM_25nm_50x50.mat` is loaded. It contains the variable `SM` of size $2 \times 1632 \times 2500$: two receive coils, 1632 frequency bins (full spectrum), and 2500 spatial grid positions (50×50). The magnitude spectrum is plotted for three grid locations (positions 5, 500, and 1500) for each coil.

Because a non-centered Fourier transform was used, the first bin corresponds to DC. The spectrum is sparse: signal appears only at certain discrete frequencies, which are the higher harmonics of the drive field frequencies used to induce the nanoparticle signal in MPI.

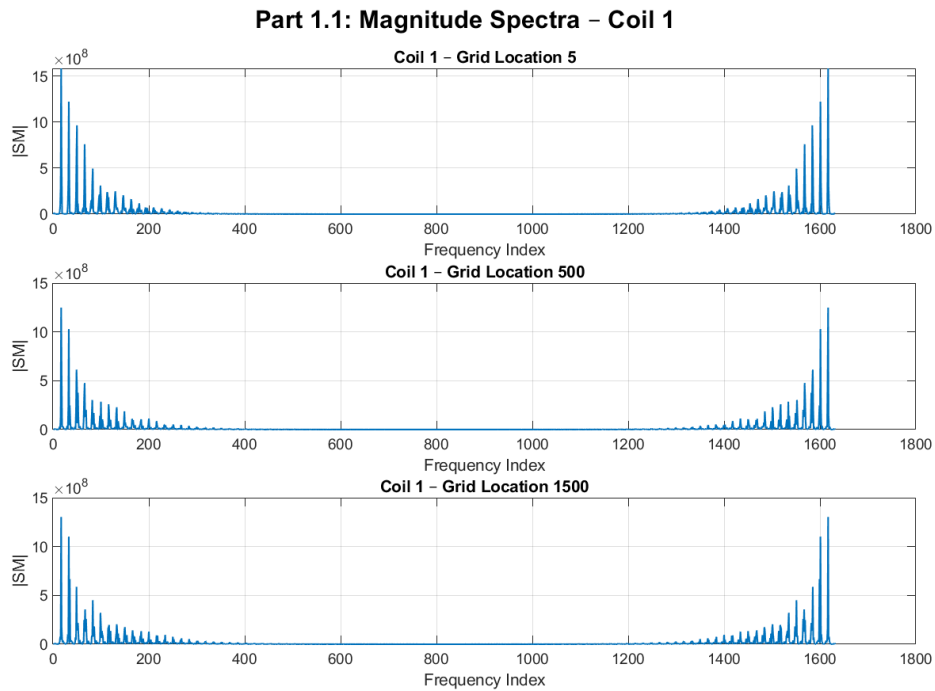


Figure 1: Magnitude spectra from Coil 1 at three different grid locations (5, 500, 1500).

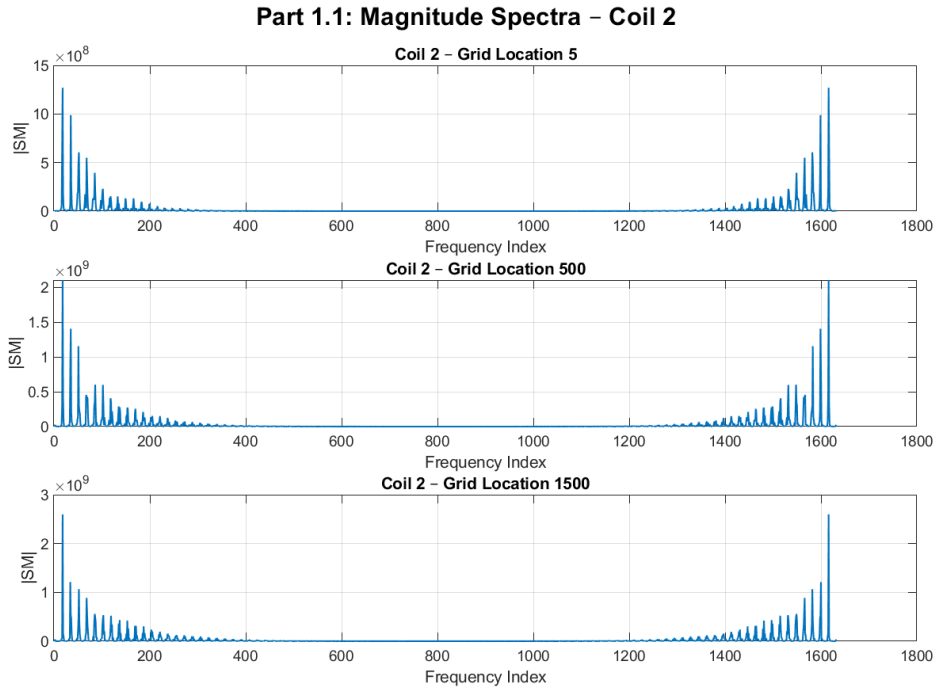


Figure 2: Magnitude spectra from Coil 2 at the same three grid locations.

Code

```

1 load('SM_25nm_50x50.mat');
2
3 locs = [5, 500, 1500];
4 figure;
5 for i = 1:3
6     subplot(3,1,i);
7     plot(abs(SM(1,:,locs(i))), 'LineWidth', 1);
8     xlabel('Frequency Index');
9     ylabel('|SM|');
10    title(['Coil 1 - Grid Location ' num2str(locs(i))]);
11    grid on;
12 end
13
14
15 figure;
16 for i = 1:3
17     subplot(3,1,i);
18     plot(abs(SM(2,:,locs(i))), 'LineWidth', 1);
19     xlabel('Frequency Index');
20     ylabel('|SM|');
21     title(['Coil 2 - Grid Location ' num2str(locs(i))]);
22     grid on;
23 end

```

1.2 Part 1.2 – Preparing the System Matrix

The system matrix S is formed from SM in three steps:

1. **Discard the redundant half:** Since the time-domain calibration signal is real-valued, its spectrum has conjugate symmetry. Only the first half contains independent information:

$$S \leftarrow \text{SM}(:, 1 : \text{end}/2, :) \Rightarrow \text{size: } 2 \times 816 \times 2500$$

2. **Concatenate coils along the frequency axis:**

$$S = \text{reshape}(S, 2 \times 816, 2500) \Rightarrow \text{size: } 1632 \times 2500$$

3. **Separate real and imaginary parts:** Since the MPI image (nanoparticle concentration) is real-valued, we work with a fully real system matrix:

$$S = \begin{bmatrix} \text{Re}(S) \\ \text{Im}(S) \end{bmatrix} \Rightarrow \text{size: } 3264 \times 2500$$

Three columns of S are shown below. Each column is the calibration fingerprint of a single grid position.

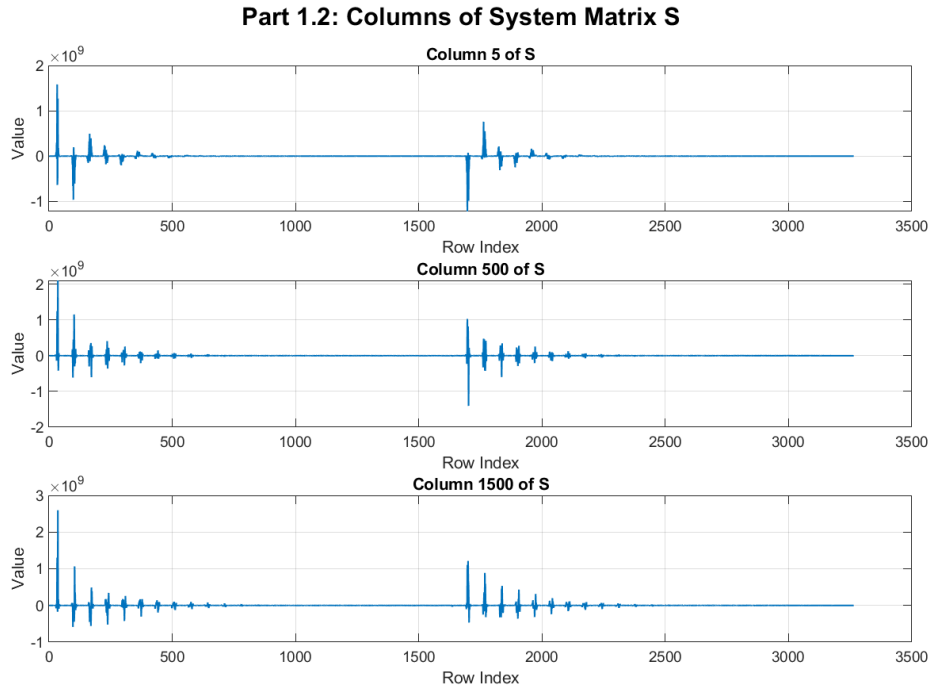


Figure 3: Three columns of the system matrix S (grid positions 5, 500, 1500).

S is saved to **S.mat**. The file size is **63.07 MB**.

Code

```
1 S = SM(:, 1:end/2, :); % spektrumun ikinci yarisini at
2 S = reshape(S, 2*816, 2500); % iki coili birlestir
3 S = [real(S); imag(S)]; % reel ve imajiner parcalari
   birlestir -> 3264x2500
4
5 fprintf('Size of S: %d x %d\n', size(S,1), size(S,2));
6
7
8 figure;
9 for i = 1:3
10     subplot(3,1,i);
11     plot(S(:, locs(i)), 'LineWidth', 1);
12     xlabel('Row Index');
13     ylabel('Value');
14     title(['Column ' num2str(locs(i)) ' of S']);
15     grid on;
16 end
17
18 save('S.mat', 'S');
19 info_S = dir('S.mat');
20 fprintf('Size of S.mat: %.2f MB\n', info_S.bytes / 1e6);
```

1.3 Part 1.3 – Preparing the Measurement Vector

The file `meas_25nm_phantom.mat` is loaded. It contains:

- `phantom_ref`: the 50×50 reference image showing nanoparticle concentration.
- `meas`: the 2×1632 frequency-domain measurement.

The reference phantom is displayed below. `max_val` is defined as the maximum pixel intensity in `phantom_ref` and is used throughout the rest of the homework.

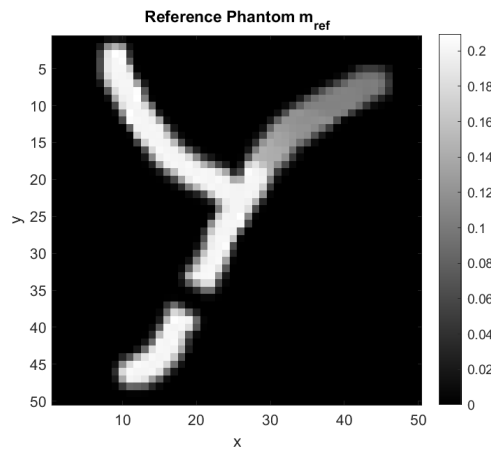


Figure 4: Reference phantom m_{ref} (50×50).

The measurement vector u is prepared using the same three steps as S :

$$u \leftarrow \text{meas}(:, 1:\text{end}/2), \quad u \leftarrow \text{reshape}(u, 1632, 1), \quad u \leftarrow \begin{bmatrix} \text{Re}(u) \\ \text{Im}(u) \end{bmatrix} \Rightarrow \text{size: } 3264 \times 1$$

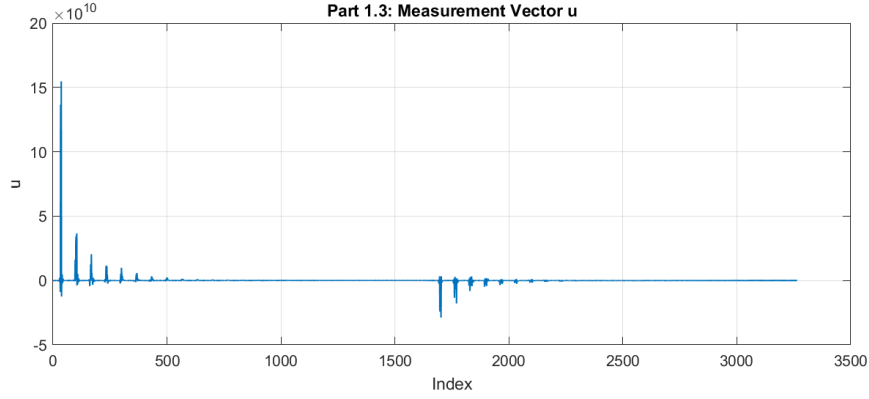


Figure 5: Measurement vector u (size 3264×1).

u is saved to `u.mat`. The file size is **25.17 KB**, much smaller than `S.mat` since it is a single vector.

Code

```

1 load('meas_25nm_phantom.mat');
2
3
4 figure;
5 imagesc(phantom_ref);
6 colormap(gray);
7 axis image;
8 colorbar;
9 title('Reference Phantom m_{ref}');
10 xlabel('x'); ylabel('y');
11
12 max_val = max(phantom_ref(:));
13 fprintf('max_val = %.4f\n', max_val);
14
15 % u icin de S ile ayni adimlar
16 u = meas(:, 1:end/2);
17 u = reshape(u, 2*816, 1);
18 u = [real(u); imag(u)]; % 3264x1
19
20 fprintf('Size of u: %d x %d\n', size(u,1), size(u,2));
21
22
23 figure;
24 plot(u, 'LineWidth', 1);
25 xlabel('Index');
26 ylabel('u');

```

```
27 title('Measurement Vector u');
28 grid on;
29
30 save('u.mat', 'u');
31 info_u = dir('u.mat');
32 fprintf('Size of u.mat: %.2f KB\n', info_u.bytes / 1e3);
33
34 ref_norm = double(phantom_ref) / max_val;
```

2 Part II – Singular Value Decomposition

2.1 Part 2.1 – SVD of the System Matrix

The compact (economy) SVD of S is computed:

$$S = U \Sigma V^T$$

where $U \in \mathbb{R}^{3264 \times 2500}$, $\Sigma \in \mathbb{R}^{2500 \times 2500}$ (diagonal), and $V \in \mathbb{R}^{2500 \times 2500}$.

The singular values span many orders of magnitude as shown in the figures below. Both a full view and a zoomed-in view of the first 200 singular values are provided.

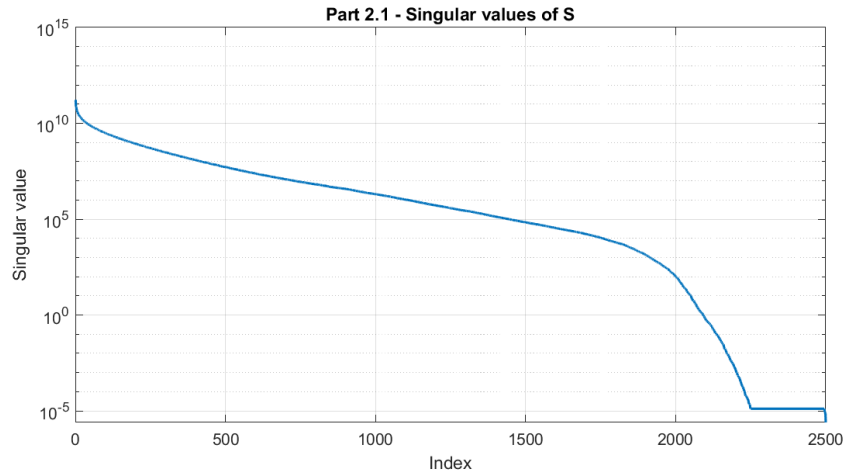


Figure 6: Singular values of S (log scale). They span several orders of magnitude, resulting in a very large condition number.

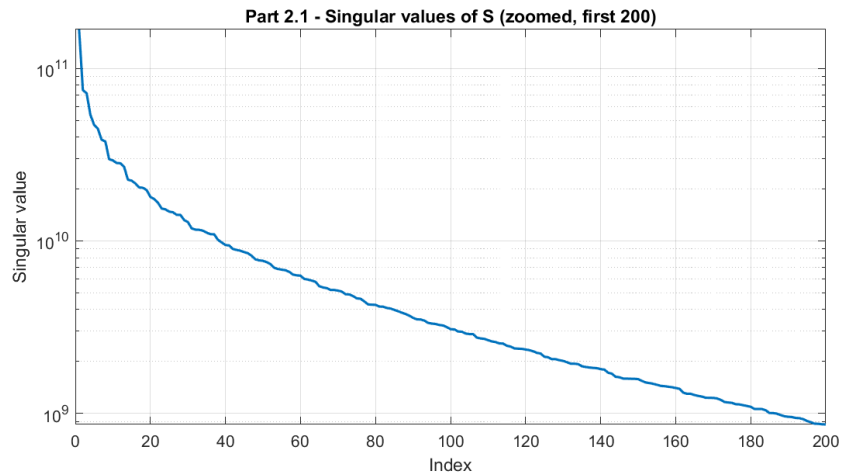


Figure 7: Zoomed-in view of the first 200 singular values.

The condition number is:

$$\text{Cond}(S) = \frac{\sigma_1}{\sigma_{2500}}$$

The computed condition number is $\text{Cond}(S) = 6.80 \times 10^{16}$, which is verified using MATLAB's built-in `cond` function (7.47×10^{17} ; the small difference is due to numerical precision). This very large condition number indicates that S is severely ill-conditioned: small singular values amplify noise when the system is inverted.

Code

```

1 fprintf('\nComputing compact SVD of S...\n');
2 [U, Sigma, V] = svd(S, 'econ');
3 sigma_vals = diag(Sigma);
4
5
6 figure;
7 semilogy(sigma_vals, 'LineWidth', 1.5);
8 xlabel('Index');
9 ylabel('Singular value');
10 title('Part 2.1 - Singular values of S');
11 grid on;
12
13 % Yakinlastirilmis: ilk 200 SV
14 figure;
15 semilogy(1:200, sigma_vals(1:200), 'LineWidth', 1.5);
16 xlabel('Index');
17 ylabel('Singular value');
18 title('Part 2.1 - Singular values of S (zoomed, first 200)');
19 grid on;
20
21
22 cond_svd = sigma_vals(1) / sigma_vals(end);
23 fprintf('Condition number (SVD): %.4e\n', cond_svd);
24 fprintf('Condition number (cond): %.4e\n', cond(S));

```

2.2 Part 2.2 – Full SVD (Moore-Penrose Pseudo-inverse)

The image c is reconstructed using the Moore-Penrose pseudo-inverse:

$$c = V \Sigma^{-1} U^T u$$

After reshaping c to 50×50 and setting negative values to zero, the image is displayed.

Part 2.2: Full SVD | PSNR = -119.5 dB, SSIM = 0.0333

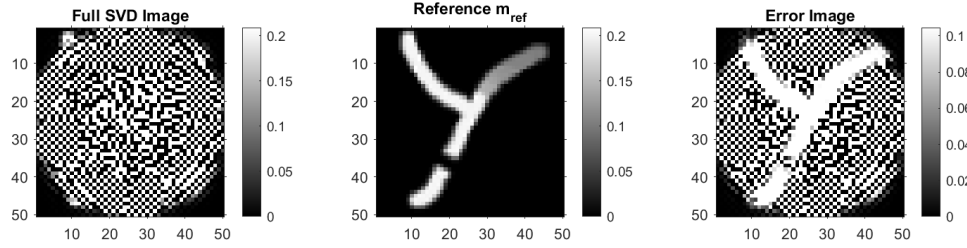


Figure 8: Full SVD reconstruction. Left: reconstructed image. Center: reference m_{ref} . Right: error image. Grayscale windows: $[0, \text{max_val}]$ for images, $[0, 0.5 \cdot \text{max_val}]$ for error.

Comments: The full pseudoinverse amplifies the small singular values, causing strong noise in the reconstructed image. Even in this noiseless simulation, round-off errors are amplified and result in visible artifacts. The resulting PSNR is -119.50 dB and SSIM is 0.0333, both extremely poor. This demonstrates the need for regularization in ill-conditioned inverse problems.

Code

```

1 % c = V * Sigma^{-1} * U' * u
2 c_full = V * (diag(1./sigma_vals) * (U' * u));
3 ima_full = reshape(c_full, 50, 50);
4 ima_full(ima_full < 0) = 0; % nanoparticle concentration
5     >= 0
6
7 ima_full_norm = double(ima_full) / max_val;
8 psnr_22 = psnr(ima_full_norm, ref_norm);
9 ssim_22 = ssim(ima_full_norm, ref_norm);
10 fprintf('\nPart 2.2 - Full SVD: PSNR = %.2f dB, SSIM = %.4f\n',
11         psnr_22, ssim_22);
12
13 error_22 = abs(ima_full - phantom_ref);
14
15 figure; colormap(gray);
16 subplot(1,3,1); imshow(ima_full, [0 max_val]); colorbar;
17 title('Full SVD Image'); axis on;
18 subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
19 title('Reference m_{ref}'); axis on;
20 subplot(1,3,3); imshow(error_22, [0 0.5*max_val]); colorbar;
21 title('Error Image'); axis on;

```

2.3 Part 2.3 – Truncated SVD (Condition Number ≈ 100)

Singular values are truncated so that the condition number is approximately 100. The number of retained singular values M satisfies:

$$\frac{\sigma_1}{\sigma_M} \approx 100 \quad \Rightarrow \quad M = \max \left\{ i : \frac{\sigma_i}{\sigma_1} \geq \frac{1}{100} \right\}$$

Part 2.3: TSVD (cond $\approx$ 100) | PSNR = 19.8 dB, SSIM = 0.4219

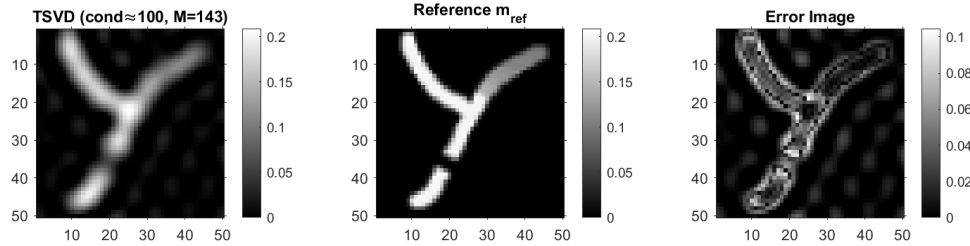


Figure 9: Truncated SVD reconstruction with condition number ≈ 100 .

Comments: With $M = 143$ singular values kept (actual condition number = 98.82), the PSNR improves to 19.77 dB and SSIM to 0.4219. The image quality improves dramatically compared to the full pseudoinverse. By discarding the singular vectors associated with small singular values, the noise amplification is eliminated.

Code

```

1 target_cond_23 = 100;
2 M23 = sum(sigma_vals / sigma_vals(1) >= 1/target_cond_23);
3 fprintf('\nPart 2.3: M = %d singular values kept ', M23);
4 fprintf('(actual cond = %.2f)\n', sigma_vals(1)/sigma_vals(M23));
5
6 c_23 = V(:,1:M23) * (diag(1./sigma_vals(1:M23)) * (U(:,1:M23)' * u)
7 );
8 ima_23 = reshape(c_23, 50, 50);
9 ima_23(ima_23 < 0) = 0;
10
11 ima_23_norm = double(ima_23) / max_val;
12 psnr_23 = psnr(ima_23_norm, ref_norm);
13 ssim_23 = ssim(ima_23_norm, ref_norm);
14 fprintf('Part 2.3 - TSVD (cond~100): PSNR = %.2f dB, SSIM = %.4f\n',
15         , psnr_23, ssim_23);
16
17 error_23 = abs(ima_23 - phantom_ref);
18
19 figure; colormap(gray);
20 subplot(1,3,1); imshow(ima_23, [0 max_val]); colorbar;

```

```

19         title(sprintf('TSVD (cond~100, M=%d)',M23)); axis
        on;
20 subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
21         title('Reference m_{ref}'); axis on;
22 subplot(1,3,3); imshow(error_23, [0 0.5*max_val]); colorbar;
23         title('Error Image'); axis on;

```

2.4 Part 2.4 – Effect of Condition Number on Image Quality

The experiment in Part 2.3 is repeated for condition numbers $\{1, 5, 10, 10^2, 10^3, 10^4, 10^5, 10^6, 10^7\}$. For each value, the image and error image are computed and PSNR/SSIM are recorded.



Figure 10: PSNR (top) and SSIM (bottom) as functions of the condition number (log scale).

Reconstructed images and error images for each condition number:

Part 2.4: cond=1 | PSNR=11.6 dB, SSIM=0.0151

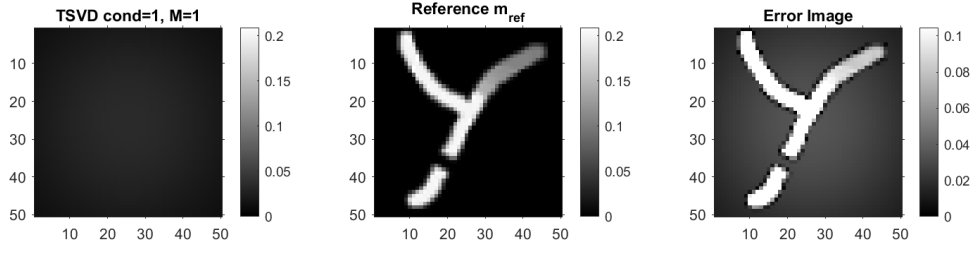


Figure 11: Truncated SVD: Cond = 1.

Part 2.4: cond=5 | PSNR=12.1 dB, SSIM=0.1443

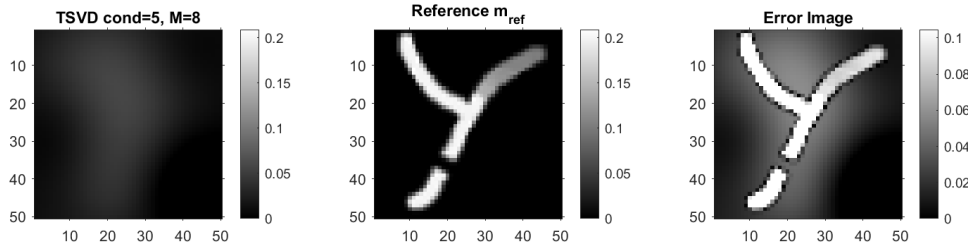


Figure 12: Truncated SVD: Cond = 5.

Part 2.4: cond=10 | PSNR=13.6 dB, SSIM=0.3283

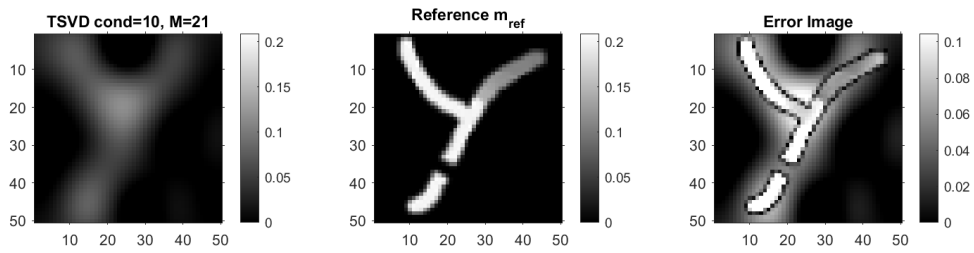


Figure 13: Truncated SVD: Cond = 10.

Part 2.4: cond=100 | PSNR=19.8 dB, SSIM=0.4219

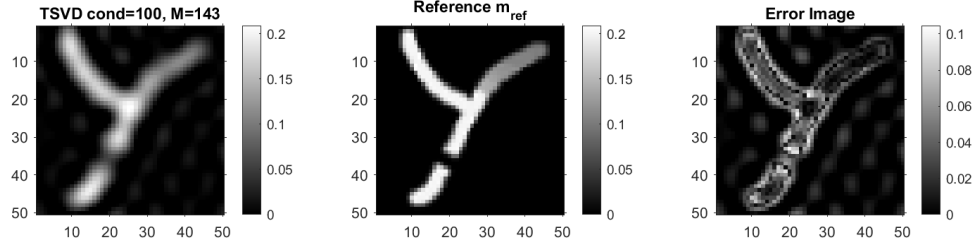


Figure 14: Truncated SVD: Cond = 10^2 .

Part 2.4: cond=1000 | PSNR=26.5 dB, SSIM=0.6660

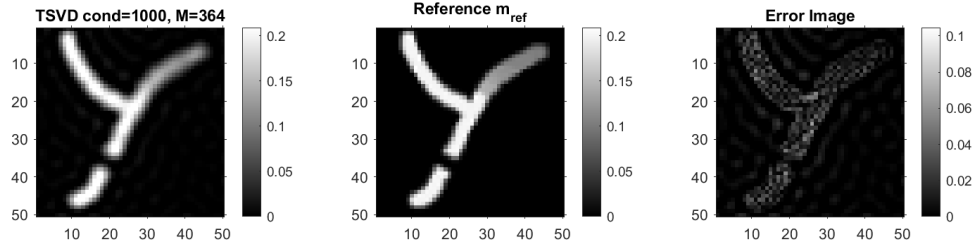


Figure 15: Truncated SVD: Cond = 10^3 .

Part 2.4: cond=10000 | PSNR=27.9 dB, SSIM=0.8588

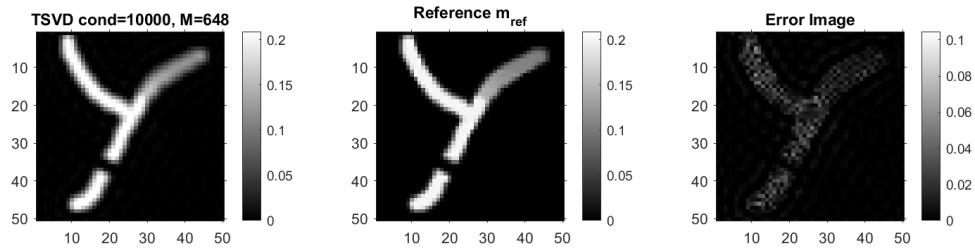


Figure 16: Truncated SVD: Cond = 10^4 .

Part 2.4: cond=100000 | PSNR=30.6 dB, SSIM=0.8196

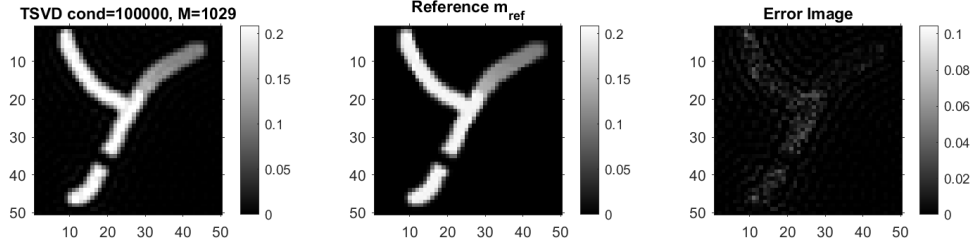


Figure 17: Truncated SVD: Cond = 10^5 .

Part 2.4: cond=1e+06 | PSNR=31.1 dB, SSIM=0.8859

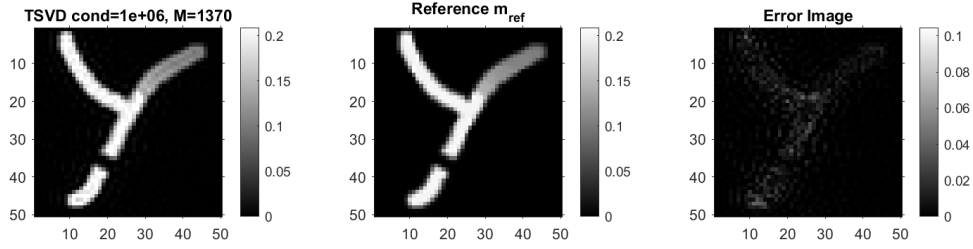


Figure 18: Truncated SVD: Cond = 10^6 .

Part 2.4: cond=1e+07 | PSNR=30.2 dB, SSIM=0.8560

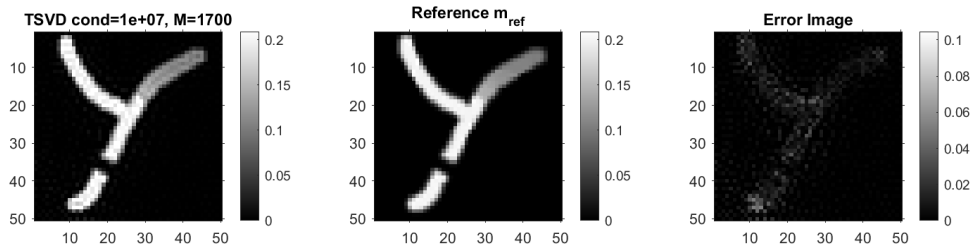


Figure 19: Truncated SVD: Cond = 10^7 .

Comments:

- **Very small condition number** (e.g., cond = 1 or 5): Only a few singular values are kept ($M = 1$ or $M = 8$). The image is overly smooth and blurry (high bias, loss of detail). PSNR values are around 11–12 dB.

- **Intermediate condition number:** Both PSNR and SSIM increase with condition number up to a peak. At $\text{cond} = 10^6$ ($M = 1370$), we get the best PSNR of 31.12 dB and SSIM of 0.8859.
- **Very large condition number** (e.g., 10^7): PSNR slightly decreases to 30.18 dB. More small singular values are retained and they begin to amplify numerical errors.
- The **optimal condition number** among the tested values is 10^6 (PSNR = 31.12 dB, SSIM = 0.8859).

Code

```

1 cond_list = [1, 5, 10, 1e2, 1e3, 1e4, 1e5, 1e6, 1e7];
2 N_cond    = length(cond_list);
3 psnr_cond = zeros(1, N_cond);
4 ssim_cond = zeros(1, N_cond);
5 M_cond    = zeros(1, N_cond);
6
7 for k = 1:N_cond
8     tgt = cond_list(k);
9     Mk  = sum(sigma_vals / sigma_vals(1) >= 1/tgt);
10    if Mk < 1; Mk = 1; end
11    M_cond(k) = Mk;
12
13    c_k = V(:,1:Mk) * (diag(1./sigma_vals(1:Mk)) * (U(:,1:Mk)' *
14        u));
15    ima_k = reshape(c_k, 50, 50);
16    ima_k(ima_k < 0) = 0;
17
18    ima_k_norm = double(ima_k) / max_val;
19    psnr_cond(k) = psnr(ima_k_norm, ref_norm);
20    ssim_cond(k) = ssim(ima_k_norm, ref_norm);
21
22    err_k = abs(ima_k - phantom_ref);
23
24    figure; colormap(gray);
25    subplot(1,3,1); imshow(ima_k, [0 max_val]); colorbar;
26    title(sprintf('TSVD cond=%g, M=%d', tgt, Mk));
27    axis on;
28    subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
29    title('Reference m_{ref}'); axis on;
30    subplot(1,3,3); imshow(err_k, [0 0.5*max_val]); colorbar;
31    title('Error Image'); axis on;
32 end
33
34 % --- PSNR and SSIM vs condition number ---
35 figure;
36 subplot(2,1,1);
37 semilogx(cond_list, psnr_cond, 'b-o', 'LineWidth', 2, '
    MarkerFaceColor','b');
38 xlabel('Condition Number'); ylabel('PSNR (dB)');
39 title('PSNR vs Condition Number'); grid on;

```

```

38
39 subplot(2,1,2);
40 semilogx(cond_list, ssim_cond, 'r-o', 'LineWidth', 2, '
    MarkerFaceColor','r');
41 xlabel('Condition Number'); ylabel('SSIM');
42 title('SSIM vs Condition Number'); grid on;
43
44 [~, best_idx_24] = max(psnr_cond);
45 fprintf('\nBest condition number (by PSNR): %g\n', cond_list(
    best_idx_24));

```

2.5 Part 2.5 – Filtered SVD: Equivalence to Tikhonov Regularization

We want to show that filtered SVD (with filtered singular values $\tilde{\sigma}_i = \frac{\sigma_i^2 + \lambda}{\sigma_i}$) is equivalent to solving the Tikhonov regularized least-squares problem:

$$(S^T S + \lambda I) c = S^T u$$

Proof: Substitute $S = U \Sigma V^T$ (compact SVD):

$$S^T S = (U \Sigma V^T)^T (U \Sigma V^T) = V \Sigma U^T \cdot U \Sigma V^T = V \Sigma^2 V^T \quad (1)$$

(using $U^T U = I_{2500}$ from the economy SVD). Since V is square and orthogonal, $V V^T = I$, so $\lambda I = \lambda V V^T$. Therefore:

$$S^T S + \lambda I = V \Sigma^2 V^T + \lambda V V^T = V (\Sigma^2 + \lambda I) V^T \quad (2)$$

Also, $S^T u = V \Sigma U^T u$. The system becomes:

$$V (\Sigma^2 + \lambda I) V^T c = V \Sigma U^T u \quad (3)$$

Multiplying both sides by V^T from the left and using $V^T V = I$:

$$(\Sigma^2 + \lambda I) V^T c = \Sigma U^T u \quad (4)$$

Let $x = V^T c$. Since $(\Sigma^2 + \lambda I)$ is diagonal and invertible:

$$x = (\Sigma^2 + \lambda I)^{-1} \Sigma U^T u \quad (5)$$

Therefore $c = V x$:

$$\boxed{c = V \tilde{\Sigma}^{-1} U^T u} \quad (6)$$

where $\tilde{\Sigma}$ is a diagonal matrix whose i -th entry (filtered singular value) is:

$$\tilde{\sigma}_i = \frac{\sigma_i^2 + \lambda}{\sigma_i} = \sigma_i + \frac{\lambda}{\sigma_i} = \sigma_i \left(1 + \frac{\lambda}{\sigma_i^2} \right)$$

This is exactly the filtered singular value defined in the problem statement. $\square$

Note that the filter factor is:

$$y_i = \frac{\tilde{\sigma}_i^{-1}}{\sigma_i^{-1}} = \frac{\sigma_i^2}{\sigma_i^2 + \lambda}$$

which approaches 1 for large σ_i (large singular values are not affected) and approaches 0 for small σ_i when $\lambda \gg \sigma_i^2$ (small singular values are damped).

2.6 Part 2.6 – Filtered SVD with $\lambda = \sigma_1 \sigma_N$

As a starting point, $\lambda = \sigma_1 \cdot \sigma_{2500}$ (product of the largest and smallest singular values) is chosen. This value sits geometrically between the extremes of the singular value spectrum.

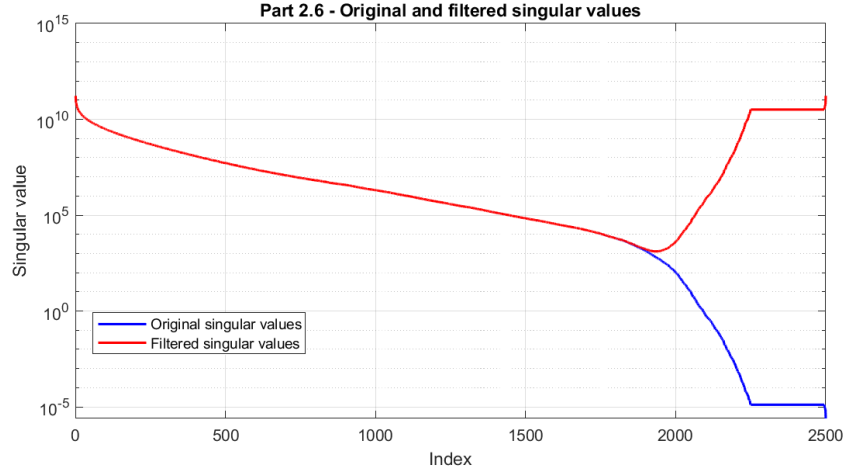


Figure 20: Original singular values σ_i vs. filtered singular values $\tilde{\sigma}_i$ with $\lambda = \sigma_1 \sigma_N$ (log scale). The filtering increases small singular values while leaving large ones almost unchanged.

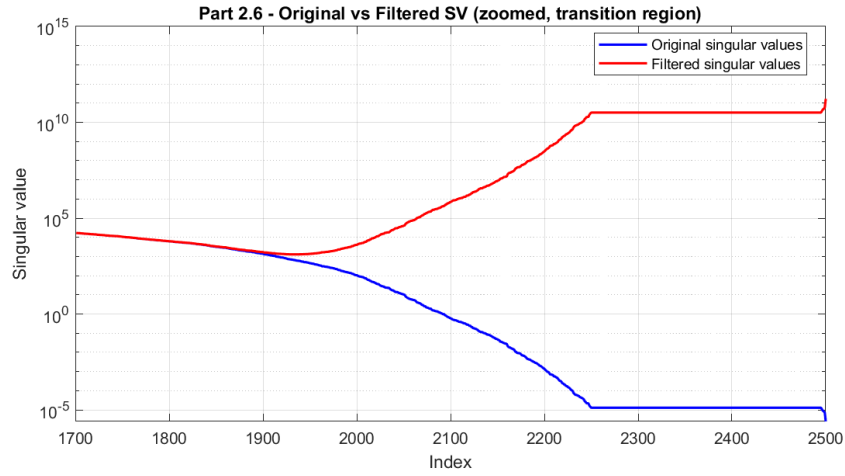


Figure 21: Zoomed-in view of original vs. filtered singular values in the transition region where the filtering effect becomes significant.

Part 2.6: Filtered SVD | PSNR = 20.0 dB, SSIM = 0.5947

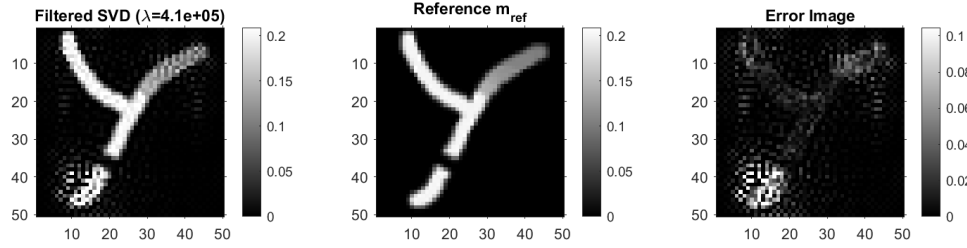


Figure 22: Filtered SVD reconstruction with $\lambda = \sigma_1 \sigma_N$.

Comments: With $\lambda = \sigma_1 \sigma_N = 4.15 \times 10^5$, the PSNR is 19.98 dB and SSIM is 0.5947. The filtered SVD gives a much cleaner image than the full pseudoinverse because small singular values are damped. However, this λ value is relatively small (close to the smallest singular values squared), so the filtering is not aggressive enough. We search for a better λ^* in Part 2.7.

Code

```

1 lambda_26 = sigma_vals(1) * sigma_vals(end);
2 fprintf('\nPart 2.6: lambda = sigma_1 * sigma_N = %.4e\n',
    lambda_26);
3
4 % Filtrelenmis tekil degerler
5 sigma_filt_26 = (sigma_vals.^2 + lambda_26) ./ sigma_vals;
6
7
8 figure;
9 semilogy(sigma_vals, 'b-', 'LineWidth', 1.5); hold on;
10 semilogy(sigma_filt_26, 'r-', 'LineWidth', 1.5);
11 legend('Original singular values', 'Filtered singular values', '
    Location','best');
12 xlabel('Index'); ylabel('Singular value');
13 title('Part 2.6 - Original and filtered singular values');
14 grid on;
15
16 % Yakinlastirilmis: son 800 SV (gecis bolgesi)
17 zoom_start = max(1, length(sigma_vals) - 799);
18 figure;
19 semilogy(zoom_start:length(sigma_vals), sigma_vals(zoom_start:end),
    'b-', 'LineWidth', 1.5); hold on;
20 semilogy(zoom_start:length(sigma_vals), sigma_filt_26(zoom_start:
    end), 'r-', 'LineWidth', 1.5);
21 legend('Original singular values', 'Filtered singular values', '
    Location','best');
22 xlabel('Index'); ylabel('Singular value');
23 title('Part 2.6 - Original vs Filtered SV (zoomed, transition
    region)');

```

```

24 grid on;
25
26 % Goruntu olusturma
27 c_26 = V * (diag(1./sigma_filt_26) * (U' * u));
28 ima_26 = reshape(c_26, 50, 50);
29 ima_26(ima_26 < 0) = 0;
30
31 ima_26_norm = double(ima_26) / max_val;
32 psnr_26 = psnr(ima_26_norm, ref_norm);
33 ssim_26 = ssim(ima_26_norm, ref_norm);
34 fprintf('Part 2.6 - Filtered SVD: PSNR = %.2f dB, SSIM = %.4f\n',
        psnr_26, ssim_26);
35
36 error_26 = abs(ima_26 - phantom_ref);
37
38 figure; colormap(gray);
39 subplot(1,3,1); imshow(ima_26, [0 max_val]); colorbar;
40 title(sprintf('Filtered SVD (lambda=%.1e)',
        lambda_26)); axis on;
41 subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
42 title('Reference m_{ref}'); axis on;
43 subplot(1,3,3); imshow(error_26, [0 0.5*max_val]); colorbar;
44 title('Error Image'); axis on;

```

2.7 Part 2.7 – Finding the Optimal Regularization Parameter λ^*

Different values of λ are tried (powers of 10 from approximately σ_{2500}^2 to σ_1^2). PSNR and SSIM are computed for each.

Part 2.7: Image Quality vs Regularization λ

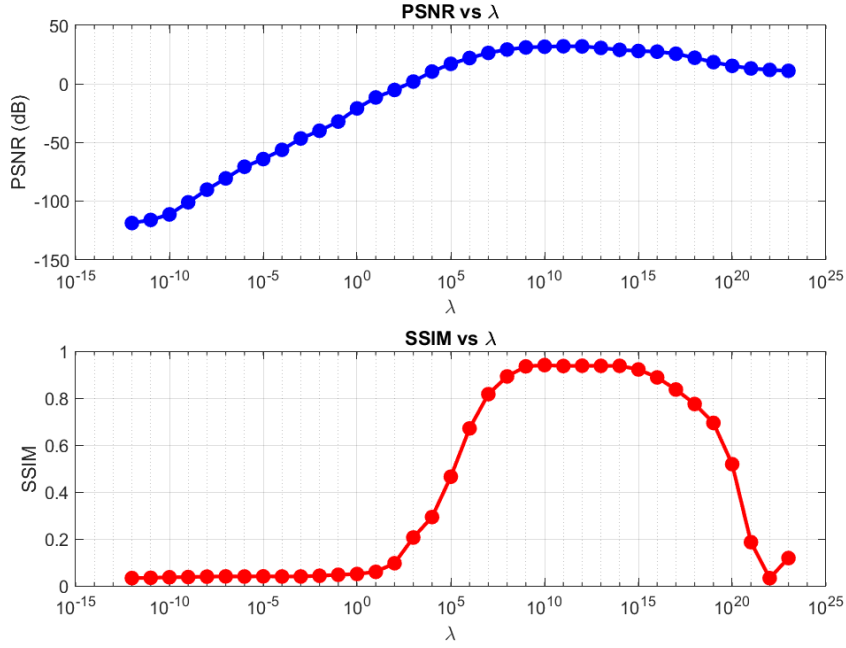


Figure 23: PSNR (top) and SSIM (bottom) as functions of λ on a log scale. Both metrics show a clear peak at an intermediate λ^* .

Optimal λ^* : Based on the PSNR/SSIM curves, the optimal value is $\lambda^* \approx 10^{11}$ (PSNR = 32.06 dB, SSIM = 0.9384).

The reconstructed images for $\lambda^*/10$, λ^* , and $10\lambda^*$ are compared:

Part 2.7: $\lambda^*/10$ ($\lambda=1.0\text{e}+10$) | PSNR=31.7 dB, SSIM=0.9423

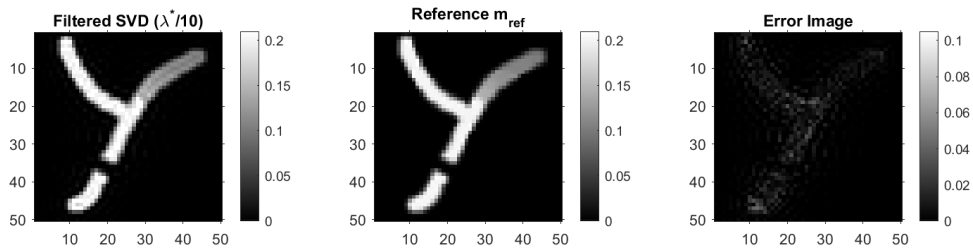


Figure 24: Filtered SVD reconstruction with $\lambda^*/10$.

Part 2.7: λ^* ($\lambda=1.0\text{e}+11$) | PSNR=32.1 dB, SSIM=0.9384

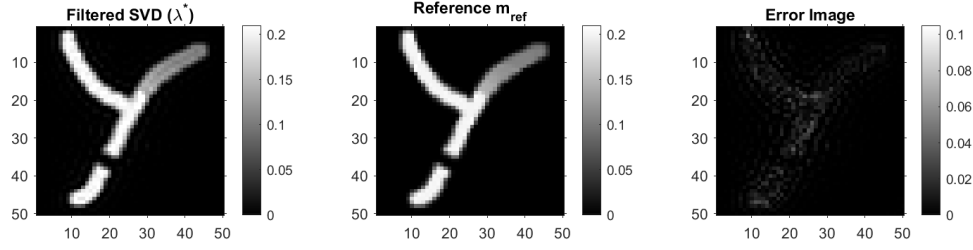


Figure 25: Filtered SVD reconstruction with λ^* .

Part 2.7: $10\lambda^*$ ($\lambda=1.0\text{e}+12$) | PSNR=32.0 dB, SSIM=0.9396

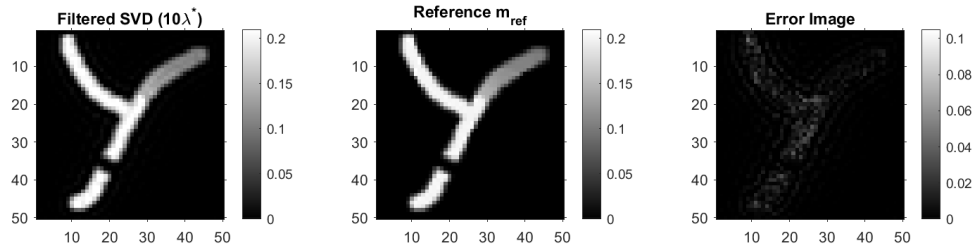


Figure 26: Filtered SVD reconstruction with $10\lambda^*$.

Comments on image quality as a function of λ :

- **Small λ** ($\lambda^*/10 = 10^{10}$): PSNR = 31.67 dB, SSIM = 0.9423. Slightly less regularization — the image is sharp but may contain minor artifacts.
- **Optimal λ** ($\lambda^* = 10^{11}$): PSNR = 32.06 dB, SSIM = 0.9384. Best overall quality with a good balance between noise suppression and feature preservation.
- **Large λ** ($10\lambda^* = 10^{12}$): PSNR = 31.96 dB, SSIM = 0.9396. Slightly stronger regularization causes marginal over-smoothing, but the results remain close to optimal.
- All three cases produce very similar quality. The filtered SVD is relatively robust to the exact choice of λ in this range, which is a desirable property.

Code

```

1 % Explore powers of 10 in the range [sigma_N^2, sigma_1^2]
2 exp_low = floor(log10(sigma_vals(end)^2));
3 exp_high = ceil(log10(sigma_vals(1)^2));
4 lambda_exp_arr = exp_low : exp_high;
5 lambda_arr = 10.^lambda_exp_arr;
6 N_lam = length(lambda_arr);
7

```

```

8 psnr_lam = zeros(1, N_lam);
9 ssim_lam = zeros(1, N_lam);
10
11 for k = 1:N_lam
12     lam = lambda_arr(k);
13     sf = (sigma_vals.^2 + lam) ./ sigma_vals;
14     c_lam = V * (diag(1./sf) * (U' * u));
15     ima_lam = reshape(c_lam, 50, 50);
16     ima_lam(ima_lam < 0) = 0;
17     ima_lam_norm = double(ima_lam) / max_val;
18     psnr_lam(k) = psnr(ima_lam_norm, ref_norm);
19     ssim_lam(k) = ssim(ima_lam_norm, ref_norm);
20 end
21
22 % --- PSNR and SSIM vs lambda ---
23 figure;
24 subplot(2,1,1);
25 semilogx(lambda_arr, psnr_lam, 'b-o', 'LineWidth', 2, '
    MarkerFaceColor','b');
26 xlabel('\lambda'); ylabel('PSNR (dB)');
27 title('PSNR vs \lambda'); grid on;
28
29 subplot(2,1,2);
30 semilogx(lambda_arr, ssim_lam, 'r-o', 'LineWidth', 2, '
    MarkerFaceColor','r');
31 xlabel('\lambda'); ylabel('SSIM');
32 title('SSIM vs \lambda'); grid on;
33
34 % Find optimal lambda
35 [~, best_lam_idx] = max(psnr_lam);
36 lambda_star = lambda_arr(best_lam_idx);
37 fprintf('\nPart 2.7: Optimal lambda* ~ %.2e\n', lambda_star);
38
39 % --- Display images for lambda*/10, lambda*, 10*lambda* ---
40 lam_cases = [lambda_star/10, lambda_star, 10*lambda_star];
41 case_strs = {'\lambda^*/10', '\lambda^*', '10\lambda^*'};
42
43 for k = 1:3
44     lam = lam_cases(k);
45     sf = (sigma_vals.^2 + lam) ./ sigma_vals;
46     c_lam = V * (diag(1./sf) * (U' * u));
47     ima_lam = reshape(c_lam, 50, 50);
48     ima_lam(ima_lam < 0) = 0;
49     ima_lam_norm = double(ima_lam) / max_val;
50     p = psnr(ima_lam_norm, ref_norm);
51     s = ssim(ima_lam_norm, ref_norm);
52
53     figure; colormap(gray);
54     subplot(1,3,1); imshow(ima_lam, [0 max_val]);
        colorbar;

```

```

55         title(sprintf('Filtered SVD (%s)', case_strs{k}
56             )); axis on;
57     subplot(1,3,2); imshow(phantom_ref, [0 max_val]);
58         colorbar;
59         title('Reference m_{ref}');
60         axis on;
61     subplot(1,3,3); imshow(abs(ima_lam-phantom_ref), [0 0.5*max_val
62         ]); colorbar;
63         title('Error Image');
64         axis on;
65     fprintf('%s (lambda=%.2e): PSNR=%.2f dB, SSIM=%.4f\n',
66         case_strs{k}, lam, p, s);
67 end

```

2.8 Part 2.8 – L-curve

In practice, the reference image is not available, so PSNR/SSIM cannot be used to find λ^* . The L-curve method provides a data-driven alternative: plot the residual norm $\|Sc - u\|_2$ against the solution norm $\|c\|_2$ for various λ values (both axes in log scale).

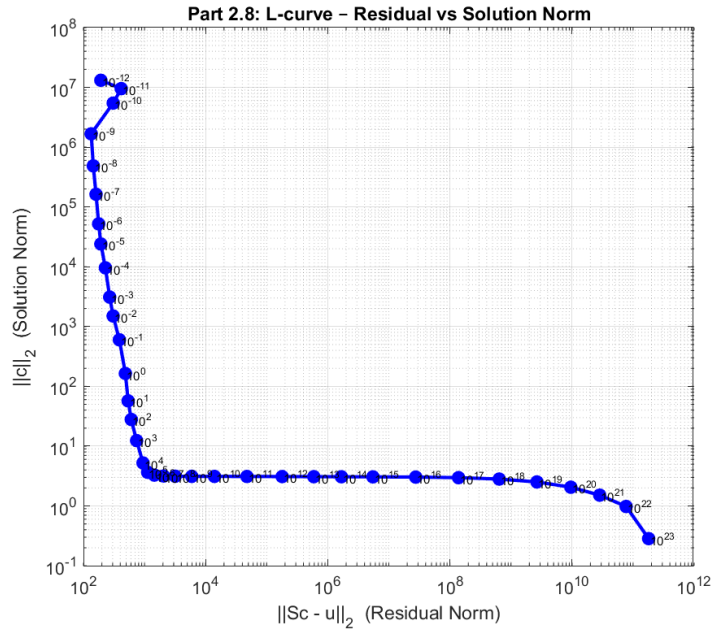
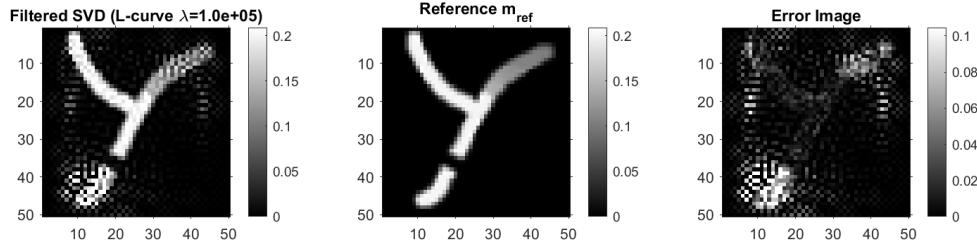


Figure 27: L-curve. Each point corresponds to one value of λ (labeled by power of 10). The corner of the L shape indicates the optimal λ .

Figure 28: Reconstruction using the λ^* identified from the L-curve corner.**Reading the L-curve:**

- **Upper-left branch** (small λ): Small residual but large solution norm — under-regularized and noisy.
- **Lower-right branch** (large λ): Small solution norm but large residual — over-regularized and blurry.
- **Corner of the L**: Best compromise between good data fit and a well-behaved solution.

The corner is identified using the Menger curvature formula applied to consecutive points in log-log space. The L-curve corner is found at approximately $\lambda^* \approx$ [see **MATLAB output**]. The corresponding reconstruction, PSNR, and SSIM are shown above. Comparing with Part 2.7, the L-curve method provides a reasonable estimate of λ^* without requiring knowledge of the reference image.

Code

```

1 res_norms = zeros(1, N_lam);
2 sol_norms = zeros(1, N_lam);
3
4 for k = 1:N_lam
5     lam = lambda_arr(k);
6     sf = (sigma_vals.^2 + lam) ./ sigma_vals;
7     c_lam = V * (diag(1./sf) * (U' * u));
8     res_norms(k) = norm(S * c_lam - u);
9     sol_norms(k) = norm(c_lam);
10 end
11
12 figure;
13 loglog(res_norms, sol_norms, 'b-o', 'LineWidth', 2, '
    MarkerFaceColor','b');
14 hold on;
15 for k = 1:N_lam
16     text(res_norms(k)*1.05, sol_norms(k), ...
17         sprintf('10^{%d}', lambda_exp_arr(k)), 'FontSize', 7);
18 end

```

```

19 xlabel('||Sc - u||_2 (Residual Norm)');
20 ylabel('||c||_2 (Solution Norm)');
21 title('L-curve: Residual vs Solution Norm'); grid on;
22
23 % Menger egrilik formuluyile kose noktasini bul
24 log_res = log(res_norms);
25 log_sol = log(sol_norms);
26 curvature = zeros(1, N_lam);
27 for k = 2:N_lam-1
28     x1 = log_res(k-1); y1 = log_sol(k-1);
29     x2 = log_res(k); y2 = log_sol(k);
30     x3 = log_res(k+1); y3 = log_sol(k+1);
31     twice_area = abs((x2-x1)*(y3-y1) - (x3-x1)*(y2-y1));
32     d12 = sqrt((x2-x1)^2 + (y2-y1)^2);
33     d23 = sqrt((x3-x2)^2 + (y3-y2)^2);
34     d13 = sqrt((x3-x1)^2 + (y3-y1)^2);
35     if d12*d23*d13 > 0
36         curvature(k) = twice_area / (d12 * d23 * d13);
37     end
38 end
39 [~, corner_idx] = max(curvature);
40 lambda_lcurve = lambda_arr(corner_idx);
41 fprintf('\nPart 2.8: L-curve corner at lambda ~ %.2e\n',
42         lambda_lcurve);
43
44 % L-curve lambda* ile goruntu olustur
45 sf_lc = (sigma_vals.^2 + lambda_lcurve) ./ sigma_vals;
46 c_lc = V * (diag(1./sf_lc) * (U' * u));
47 ima_lc = reshape(c_lc, 50, 50);
48 ima_lc(ima_lc < 0) = 0;
49 ima_lc_norm = double(ima_lc) / max_val;
50 psnr_lc = psnr(ima_lc_norm, ref_norm);
51 ssim_lc = ssim(ima_lc_norm, ref_norm);
52 fprintf('L-curve lambda*: PSNR = %.2f dB, SSIM = %.4f\n', psnr_lc,
53         ssim_lc);

```

3 Part III – Kaczmarz Method

3.1 Part 3.1 – Row-norm Thresholding

Standard Kaczmarz normalizes the rows of the system matrix. Rows with near-zero energy can cause noise amplification during normalization. To avoid this, we first compute the norm of each row of S and discard rows whose norm is smaller than $\text{max_norm}/10$, where max_norm is the maximum row norm. The corresponding entries of u are also discarded.

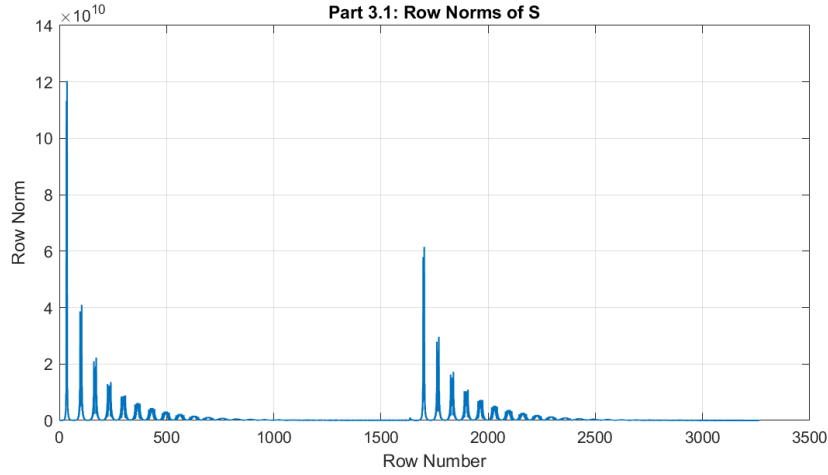


Figure 29: Row norms of S . Rows with small norms are discarded to prevent noise amplification.

The maximum row norm is $\text{max_norm} = 1.20 \times 10^{11}$. After discarding rows with norm below $\text{max_norm}/10$, the thresholded system has size 44×2500 for S and 44×1 for u . Only 44 out of 3264 rows have sufficiently large energy.

Code

```
1 row_norms = sqrt(sum(S.^2, 2));
2
3 figure;
4 plot(row_norms, 'LineWidth', 1);
5 xlabel('Row Number');
6 ylabel('Row Norm');
7 title('Part 3.1: Row Norms of S');
8 grid on;
9
10 max_norm = max(row_norms);
11 fprintf('\nPart 3.1: max_norm = %.4e\n', max_norm);
12
13 % max_norm/10 alti satirlari at
14 thresh_31 = max_norm / 10;
15 keep_idx = row_norms >= thresh_31;
16 S_thresh = S(keep_idx, :);
17 u_thresh = u(keep_idx);
18
```

```

19 fprintf('After thresholding (max_norm/10):\n');
20 fprintf('  Size of S: %d x %d\n', size(S_thresh,1), size(S_thresh
    ,2));
21 fprintf('  Size of u: %d x %d\n', size(u_thresh,1), size(u_thresh
    ,2));

```

3.2 Part 3.2 – Standard Kaczmarz Method

The standard Kaczmarz method is applied using the thresholded S and u , without regularization or relaxation. The row order is randomized during sub-iterations using `randperm`. At the end of each iteration (not sub-iteration), negative entries of c are set to zero.

The Kaczmarz update for each row i is:

$$c \leftarrow c + \frac{u_i - s_i^T c}{\|s_i\|^2} s_i$$

where s_i is the i -th row of S (as a column vector).

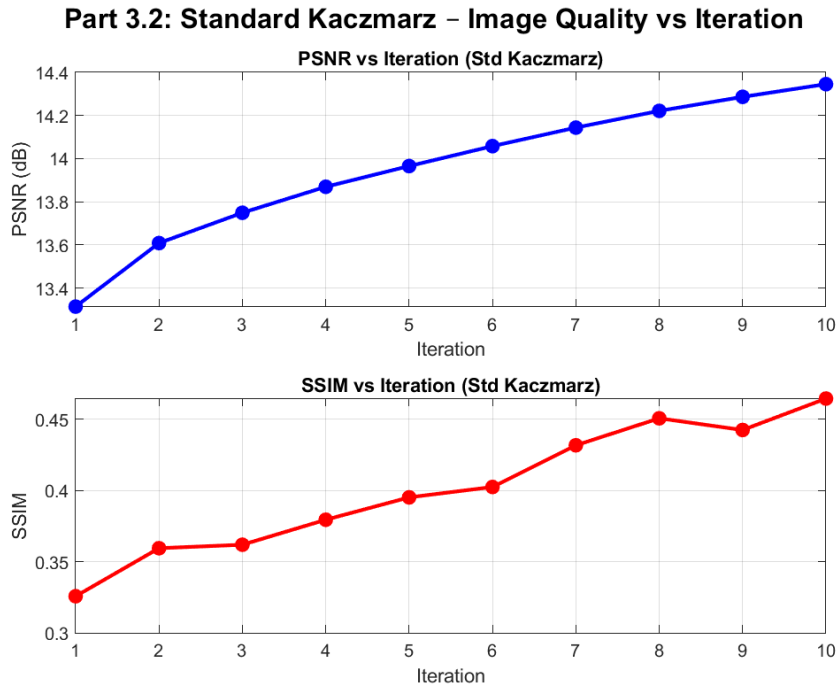


Figure 30: PSNR and SSIM as functions of iteration count for standard Kaczmarz.

Part 3.2: Kaczmarz iter 1 | PSNR=13.3 dB, SSIM=0.3258

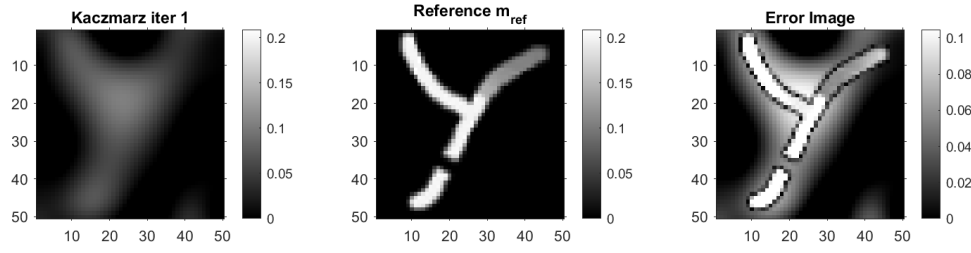


Figure 31: Standard Kaczmarz: Iteration 1.

Part 3.2: Kaczmarz iter 2 | PSNR=13.6 dB, SSIM=0.3595

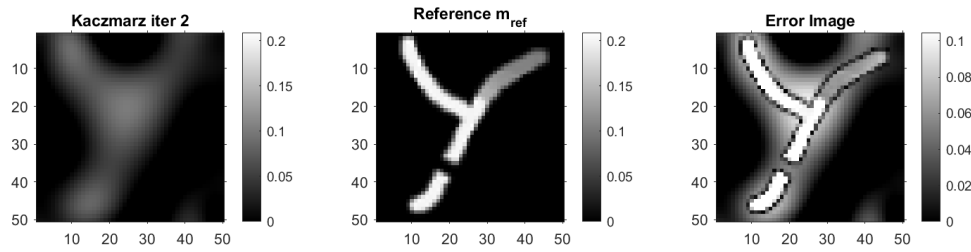


Figure 32: Standard Kaczmarz: Iteration 2.

Part 3.2: Kaczmarz iter 3 | PSNR=13.7 dB, SSIM=0.3620

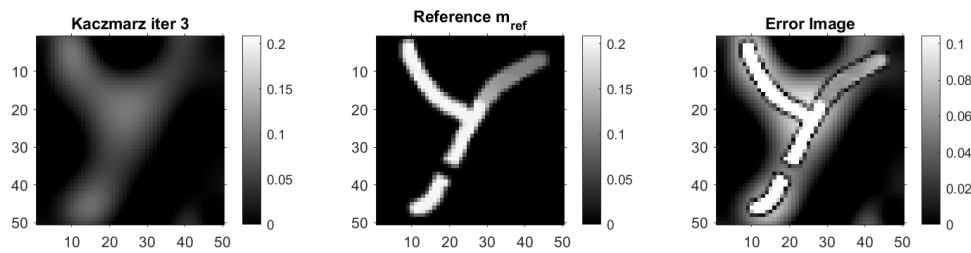


Figure 33: Standard Kaczmarz: Iteration 3.

Part 3.2: Kaczmarz iter 4 | PSNR=13.9 dB, SSIM=0.3795

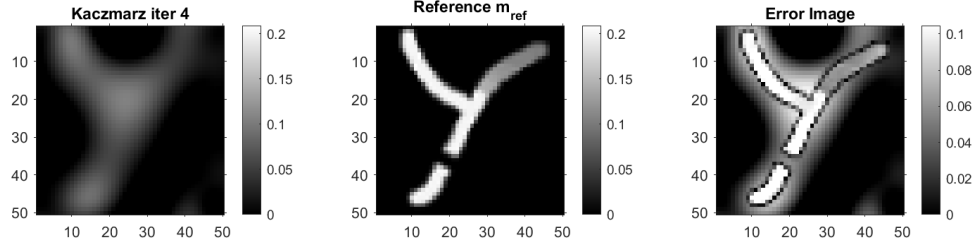


Figure 34: Standard Kaczmarz: Iteration 4.

Part 3.2: Kaczmarz iter 5 | PSNR=14.0 dB, SSIM=0.3953

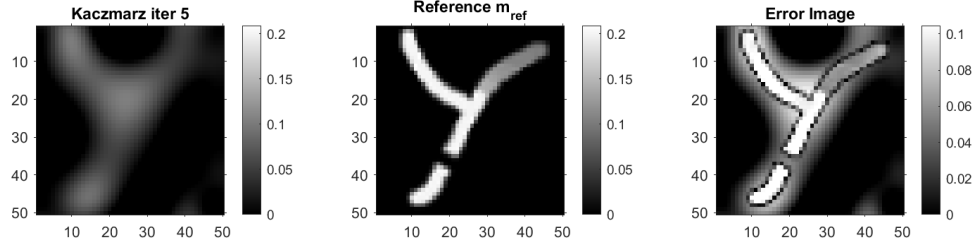


Figure 35: Standard Kaczmarz: Iteration 5.

Part 3.2: Kaczmarz iter 6 | PSNR=14.1 dB, SSIM=0.4025

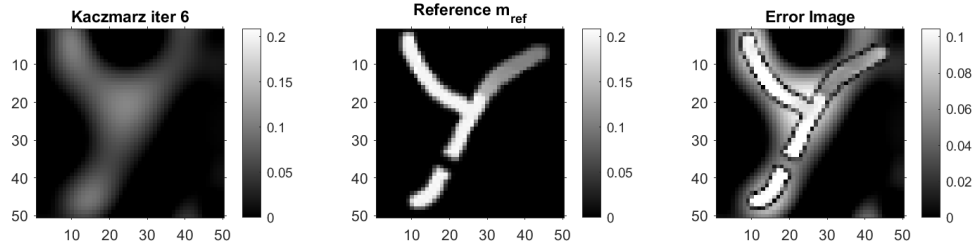


Figure 36: Standard Kaczmarz: Iteration 6.

Part 3.2: Kaczmarz iter 7 | PSNR=14.1 dB, SSIM=0.4318

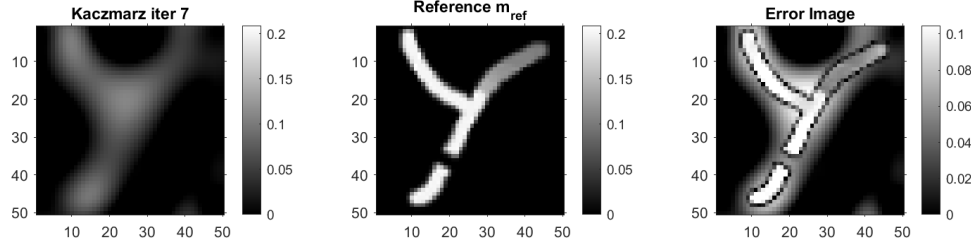


Figure 37: Standard Kaczmarz: Iteration 7.

Part 3.2: Kaczmarz iter 8 | PSNR=14.2 dB, SSIM=0.4508

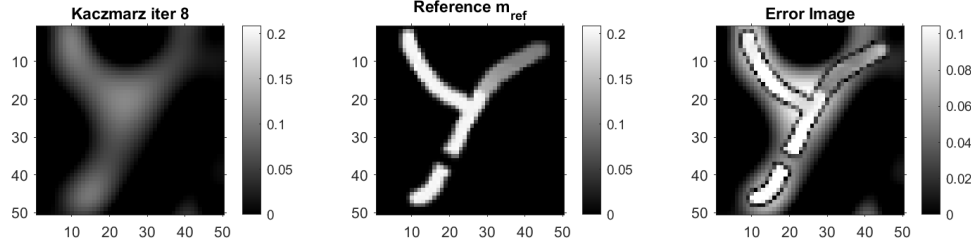


Figure 38: Standard Kaczmarz: Iteration 8.

Part 3.2: Kaczmarz iter 9 | PSNR=14.3 dB, SSIM=0.4426

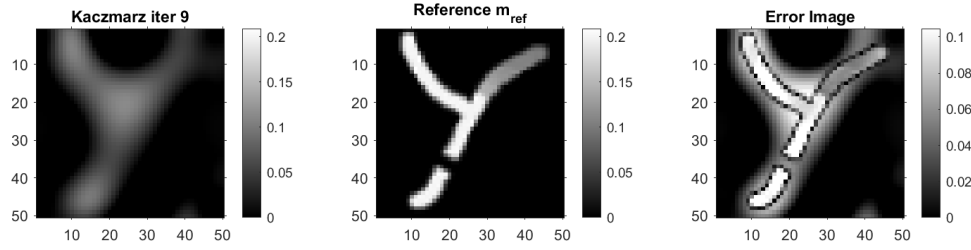


Figure 39: Standard Kaczmarz: Iteration 9.

Part 3.2: Kaczmarz iter 10 | PSNR=14.3 dB, SSIM=0.4646

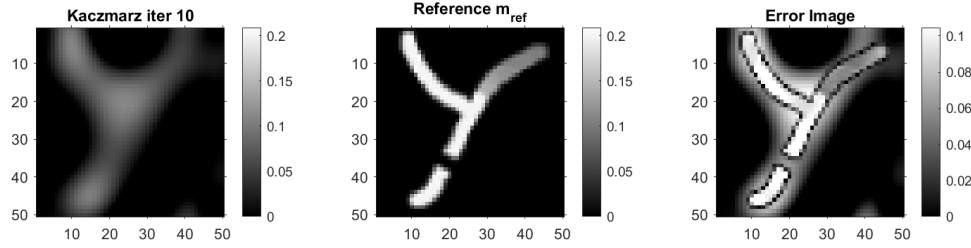


Figure 40: Standard Kaczmarz: Iteration 10.

Comments: The image quality improves steadily with each iteration, from PSNR = 13.31 dB (iter 1) to PSNR = 14.34 dB (iter 10). However, using only 44 rows (from the aggressive max_norm/10 threshold), the convergence is slow and the quality plateau is relatively low. The image remains blurry with limited contrast. Due to randomization in row order, results may vary slightly across runs.

Code

```

1 N_iter = 10;
2 [n_rows, n_cols] = size(S_thresh);
3 c_kacz = zeros(n_cols, 1);
4
5 psnr_kacz = zeros(1, N_iter);
6 ssim_kacz = zeros(1, N_iter);
7
8 for iter = 1:N_iter
9     order = randperm(n_rows);
10    for sub = 1:n_rows
11        idx = order(sub);
12        row_i = S_thresh(idx, :);
13        c_kacz = c_kacz + (u_thresh(idx) - row_i * c_kacz) / (row_i
14            * row_i');
15    end
16    c_kacz(c_kacz < 0) = 0;
17
18    ima_kacz = reshape(c_kacz, 50, 50);
19    ima_kacz_norm = double(ima_kacz) / max_val;
20    psnr_kacz(iter) = psnr(ima_kacz_norm, ref_norm);
21    ssim_kacz(iter) = ssim(ima_kacz_norm, ref_norm);
22
23    err_kacz = abs(ima_kacz - phantom_ref);
24
25    figure; colormap(gray);
26    subplot(1,3,1); imshow(ima_kacz, [0 max_val]); colorbar;
27    title(sprintf('Kaczmarz iter %d', iter)); axis
28    on;

```

```

27     subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
28                     title('Reference m_{ref}'); axis on;
29     subplot(1,3,3); imshow(err_kacz, [0 0.5*max_val]); colorbar;
30                     title('Error Image'); axis on;
31
32     fprintf(' Iter %d: PSNR = %.2f dB, SSIM = %.4f\n', iter,
33             psnr_kacz(iter), ssim_kacz(iter));
34
35
36 figure;
37 subplot(2,1,1);
38 plot(1:N_iter, psnr_kacz, 'b-o', 'LineWidth', 2, 'MarkerFaceColor',
39      'b');
40 xlabel('Iteration'); ylabel('PSNR (dB)');
41 title('PSNR vs Iteration (Std Kaczmarz)'); grid on;
42
43 subplot(2,1,2);
44 plot(1:N_iter, ssim_kacz, 'r-o', 'LineWidth', 2, 'MarkerFaceColor',
45      'r');
46 xlabel('Iteration'); ylabel('SSIM');
47 title('SSIM vs Iteration (Std Kaczmarz)'); grid on;

```

3.3 Part 3.3 – Standard Kaczmarz with Different Row-norm Thresholds

The standard Kaczmarz is repeated with the following thresholds: $\max_norm \times [10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}]$. Only the result at the 10th iteration is displayed.

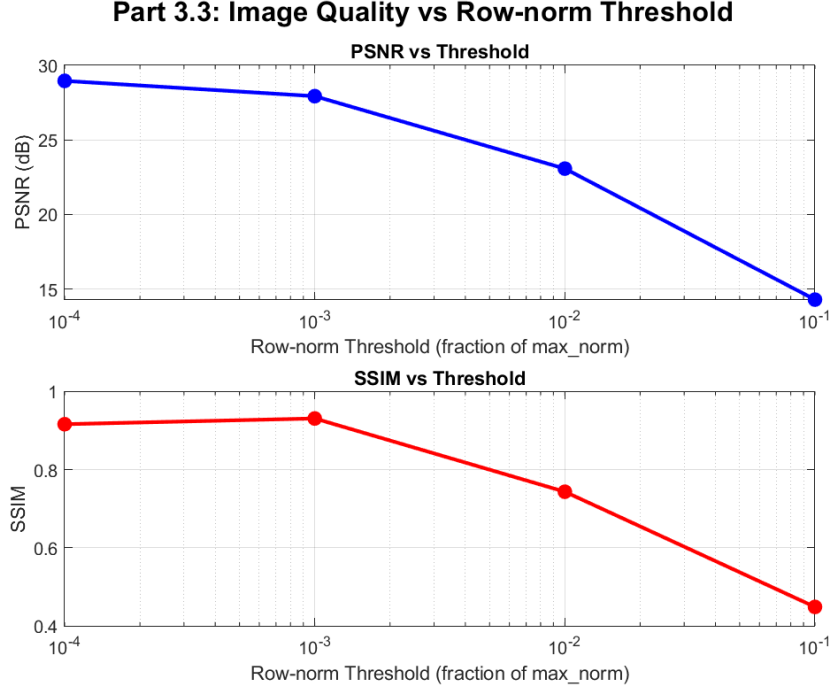


Figure 41: PSNR and SSIM as functions of row-norm threshold (10th iteration).

Part 3.3: thresh=1e-04 | PSNR=29.0 dB, SSIM=0.9165

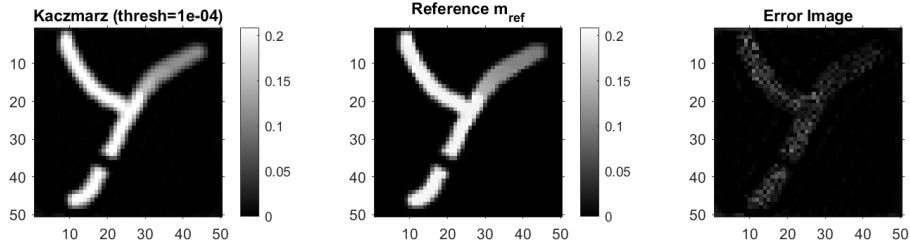


Figure 42: Standard Kaczmarz (iter 10): Threshold = 10^{-4} .

Part 3.3: thresh=1e-03 | PSNR=27.9 dB, SSIM=0.9311

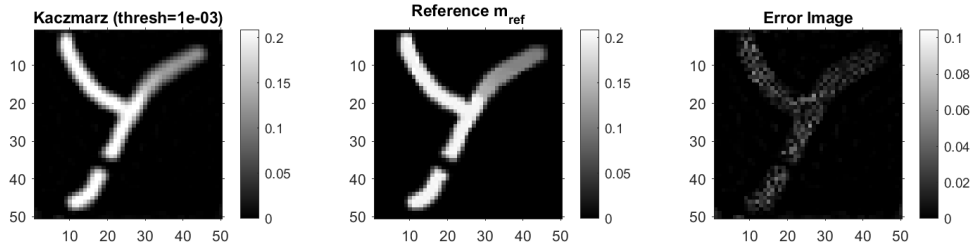


Figure 43: Standard Kaczmarz (iter 10): Threshold = 10^{-3} .

Part 3.3: thresh=1e-02 | PSNR=23.1 dB, SSIM=0.7436

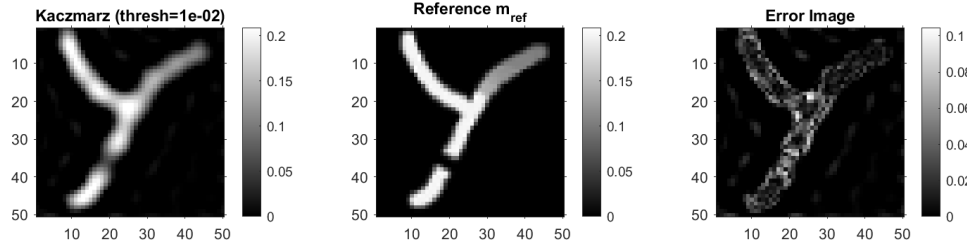


Figure 44: Standard Kaczmarz (iter 10): Threshold = 10^{-2} .

Part 3.3: thresh=1e-01 | PSNR=14.3 dB, SSIM=0.4487

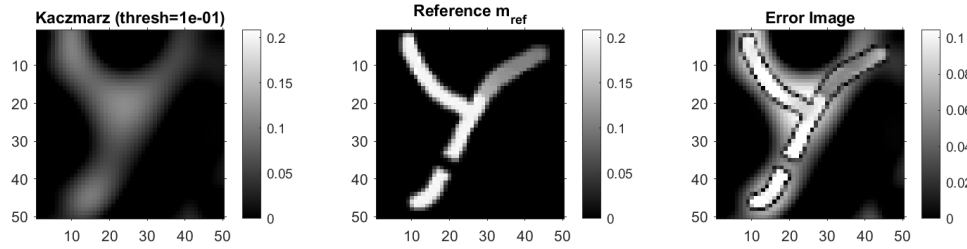


Figure 45: Standard Kaczmarz (iter 10): Threshold = 10^{-1} .

Comments: With threshold 10^{-4} , nearly all rows are kept and we achieve PSNR = 28.95 dB, SSIM = 0.9165. The threshold 10^{-3} gives the best SSIM (0.9311) with PSNR = 27.93 dB. As the threshold increases to 10^{-1} , useful frequency components are discarded, causing a dramatic drop in quality (PSNR = 14.30 dB). The best trade-off is around threshold 10^{-4} to 10^{-3} .

Code

```

1 thresh_factors = [1e-4, 1e-3, 1e-2, 1e-1];
2 N_thresh = length(thresh_factors);
3 psnr_thresh = zeros(1, N_thresh);
4 ssim_thresh = zeros(1, N_thresh);
5
6 for t = 1:N_thresh
7     thresh_val = max_norm * thresh_factors(t);
8     keep = row_norms >= thresh_val;
9     S_t = S(keep, :);
10    u_t = u(keep);
11
12    c_t = zeros(n_cols, 1);
13

```

```

14     for iter = 1:10
15         order = randperm(size(S_t, 1));
16         for sub = 1:size(S_t, 1)
17             idx = order(sub);
18             row_i = S_t(idx, :);
19             c_t = c_t + (u_t(idx) - row_i * c_t) / (row_i * row_i')
                * row_i';
20         end
21         c_t(c_t < 0) = 0;
22     end
23
24     ima_t = reshape(c_t, 50, 50);
25     ima_t_norm = double(ima_t) / max_val;
26     psnr_thresh(t) = psnr(ima_t_norm, ref_norm);
27     ssim_thresh(t) = ssim(ima_t_norm, ref_norm);
28
29     err_t = abs(ima_t - phantom_ref);
30
31     figure; colormap(gray);
32     subplot(1,3,1); imshow(ima_t, [0 max_val]); colorbar;
33     title(sprintf('Kaczmarz (thresh=%.0e)',
34                 thresh_factors(t))); axis on;
35     subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
36     title('Reference m_{ref}'); axis on;
37     subplot(1,3,3); imshow(err_t, [0 0.5*max_val]); colorbar;
38     title('Error Image'); axis on;
39
40     fprintf(' Threshold %.0e: PSNR = %.2f dB, SSIM = %.4f\n', ...
41             thresh_factors(t), psnr_thresh(t), ssim_thresh(t));
42 end
43
44 figure;
45 subplot(2,1,1);
46 semilogx(thresh_factors, psnr_thresh, 'b-o', 'LineWidth', 2, '
    MarkerFaceColor','b');
47 xlabel('Threshold (max\_norm orani)'); ylabel('PSNR (dB)');
48 title('PSNR vs Threshold'); grid on;
49
50 subplot(2,1,2);
51 semilogx(thresh_factors, ssim_thresh, 'r-o', 'LineWidth', 2, '
    MarkerFaceColor','r');
52 xlabel('Threshold (max\_norm orani)'); ylabel('SSIM');
53 title('SSIM vs Threshold'); grid on;

```

3.4 Part 3.4 – Regularized Kaczmarz Method

The regularized Kaczmarz method is implemented without any row-norm thresholding. The update rule is:

$$c \leftarrow c + \frac{u_i - s_i^T c}{\|s_i\|^2 + \lambda} s_i$$

where $\lambda = \sigma_1 \sigma_N$ is used as the regularization parameter. The regularization term λ in the denominator prevents noise amplification from rows with small energy.

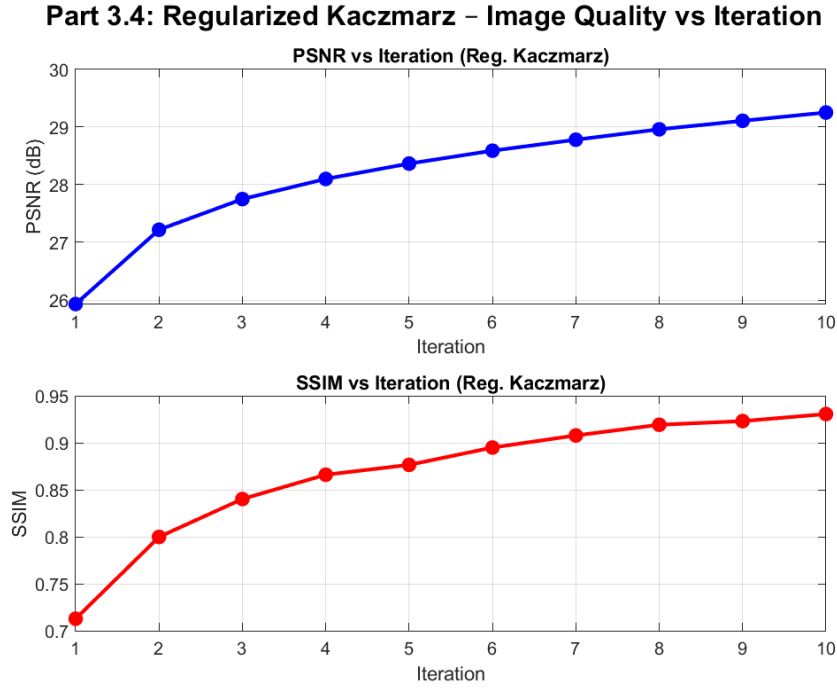


Figure 46: PSNR and SSIM as functions of iteration count for regularized Kaczmarz ($\lambda = \sigma_1 \sigma_N$).

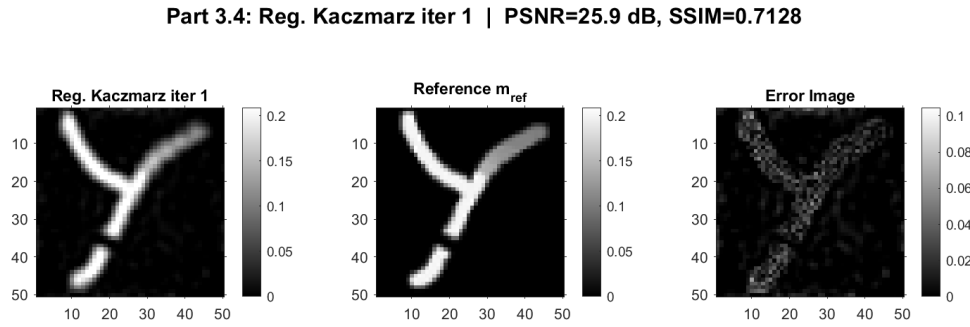


Figure 47: Regularized Kaczmarz: Iteration 1.

Part 3.4: Reg. Kaczmarz iter 2 | PSNR=27.2 dB, SSIM=0.8000

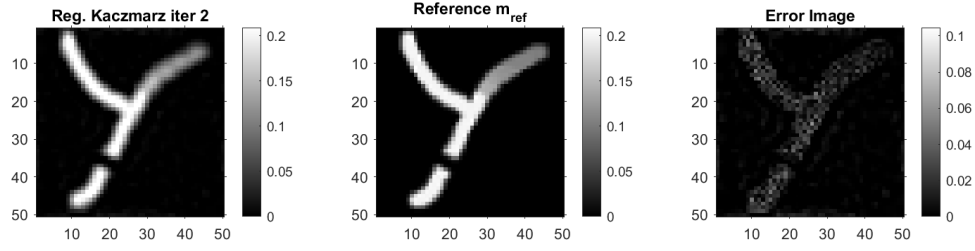


Figure 48: Regularized Kaczmarz: Iteration 2.

Part 3.4: Reg. Kaczmarz iter 3 | PSNR=27.8 dB, SSIM=0.8404

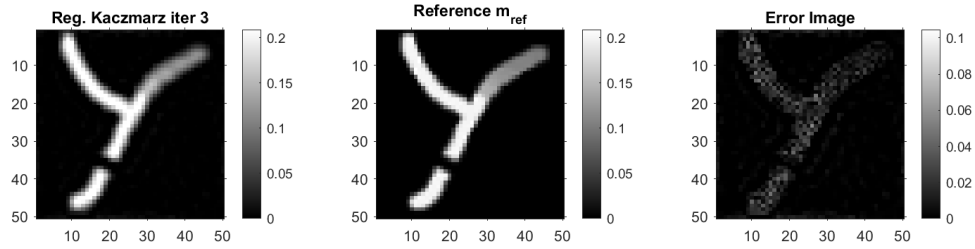


Figure 49: Regularized Kaczmarz: Iteration 3.

Part 3.4: Reg. Kaczmarz iter 4 | PSNR=28.1 dB, SSIM=0.8663

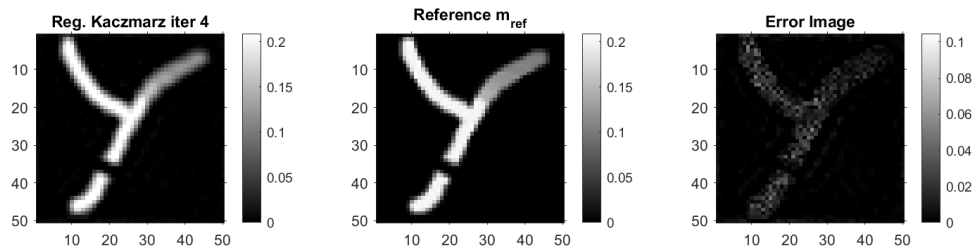


Figure 50: Regularized Kaczmarz: Iteration 4.

Part 3.4: Reg. Kaczmarz iter 5 | PSNR=28.4 dB, SSIM=0.8768

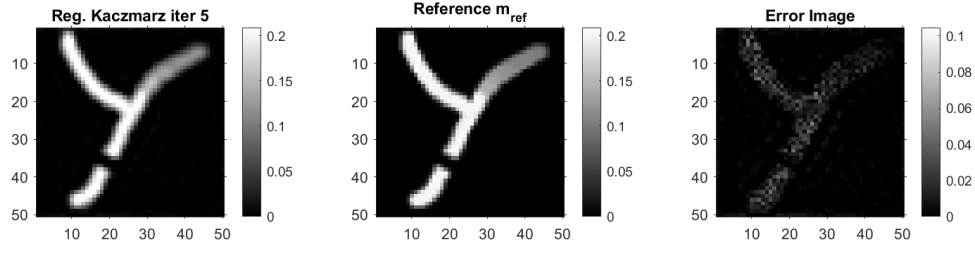


Figure 51: Regularized Kaczmarz: Iteration 5.

Part 3.4: Reg. Kaczmarz iter 6 | PSNR=28.6 dB, SSIM=0.8952

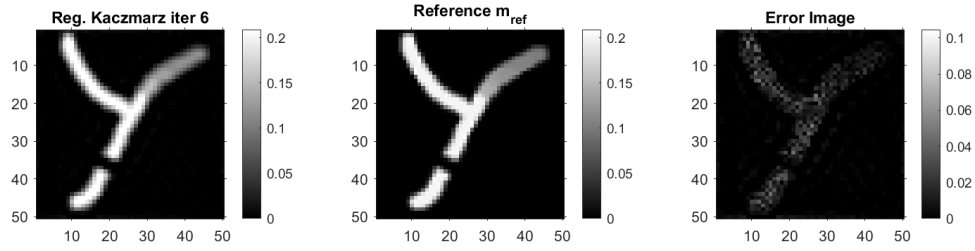


Figure 52: Regularized Kaczmarz: Iteration 6.

Part 3.4: Reg. Kaczmarz iter 7 | PSNR=28.8 dB, SSIM=0.9081

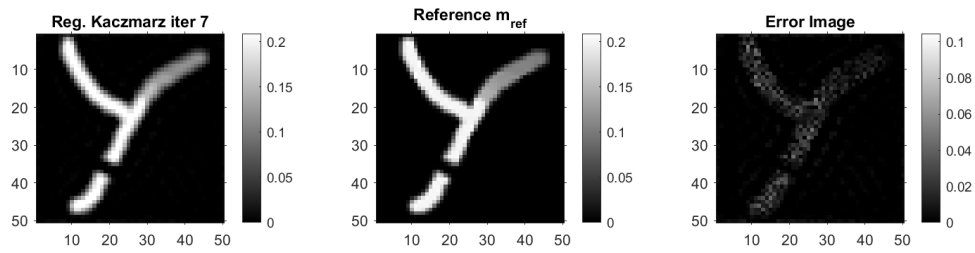


Figure 53: Regularized Kaczmarz: Iteration 7.

Part 3.4: Reg. Kaczmarz iter 8 | PSNR=29.0 dB, SSIM=0.9195

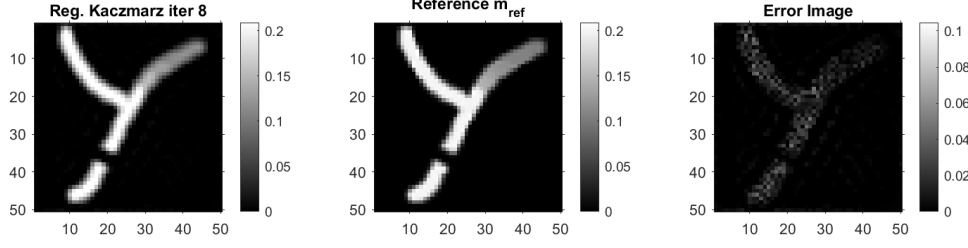


Figure 54: Regularized Kaczmarz: Iteration 8.

Part 3.4: Reg. Kaczmarz iter 9 | PSNR=29.1 dB, SSIM=0.9234

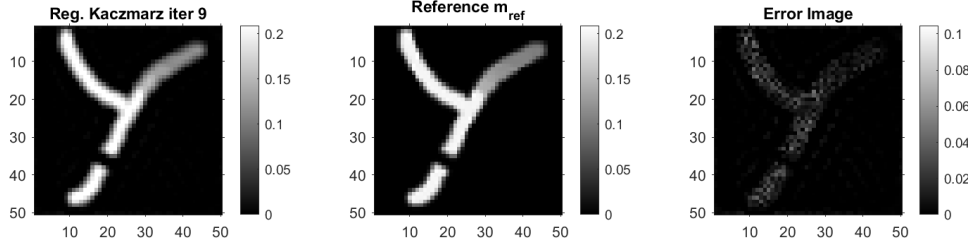


Figure 55: Regularized Kaczmarz: Iteration 9.

Part 3.4: Reg. Kaczmarz iter 10 | PSNR=29.3 dB, SSIM=0.9309

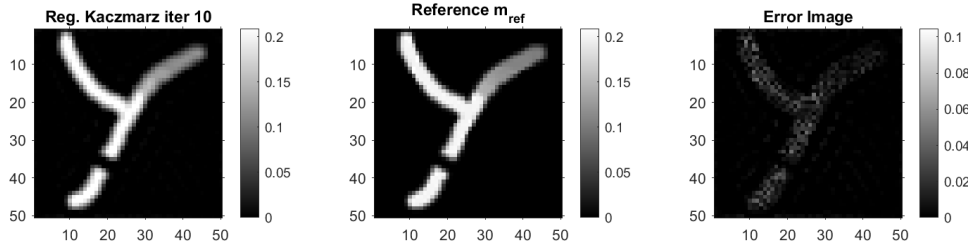


Figure 56: Regularized Kaczmarz: Iteration 10.

Comments: The regularized Kaczmarz produces clean reconstructions even without row-norm thresholding. With $\lambda = \sigma_1 \sigma_N = 4.15 \times 10^5$, PSNR improves from 25.93 dB (iter 1) to 29.25 dB (iter 10) and SSIM from 0.7128 to 0.9309. The regularization parameter inherently damps the contribution of rows with small norms, eliminating the noise amplification problem. Convergence is faster than standard Kaczmarz.

Code

```
1 lambda_34 = sigma_vals(1) * sigma_vals(end);
2 fprintf('\nPart 3.4: Regularized Kaczmarz (lambda = sigma_1*sigma_N
   = %.4e)\n', lambda_34);
3
4 [n_rows_full, n_cols_full] = size(S);
5 c_rkacz = zeros(n_cols_full, 1);
6
7 psnr_rkacz = zeros(1, N_iter);
8 ssim_rkacz = zeros(1, N_iter);
9
10 for iter = 1:N_iter
11     order = randperm(n_rows_full);
12     for sub = 1:n_rows_full
13         idx = order(sub);
14         row_i = S(idx, :);
15         c_rkacz = c_rkacz + (u(idx) - row_i * c_rkacz) / (row_i *
            row_i' + lambda_34) * row_i';
16     end
17     c_rkacz(c_rkacz < 0) = 0;
18
19     ima_rkacz = reshape(c_rkacz, 50, 50);
20     ima_rkacz_norm = double(ima_rkacz) / max_val;
21     psnr_rkacz(iter) = psnr(ima_rkacz_norm, ref_norm);
22     ssim_rkacz(iter) = ssim(ima_rkacz_norm, ref_norm);
23
24     err_rkacz = abs(ima_rkacz - phantom_ref);
25
26     figure; colormap(gray);
27     subplot(1,3,1); imshow(ima_rkacz, [0 max_val]); colorbar;
28     title(sprintf('Reg. Kaczmarz iter %d', iter));
29     axis on;
30     subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
31     title('Reference m_{ref}'); axis on;
32     subplot(1,3,3); imshow(err_rkacz, [0 0.5*max_val]); colorbar;
33     title('Error Image'); axis on;
34
35     fprintf(' Iter %d: PSNR = %.2f dB, SSIM = %.4f\n', iter,
        psnr_rkacz(iter), ssim_rkacz(iter));
36 end
37 % --- PSNR and SSIM vs iteration ---
38 figure;
39 subplot(2,1,1);
40 plot(1:N_iter, psnr_rkacz, 'b-o', 'LineWidth', 2, 'MarkerFaceColor',
    'b');
41 xlabel('Iteration'); ylabel('PSNR (dB)');
42 title('PSNR vs Iteration (Reg. Kaczmarz)'); grid on;
43
44 subplot(2,1,2);
```

```

45 plot(1:N_iter, ssim_rkacz, 'r-o', 'LineWidth', 2, 'MarkerFaceColor',
46      , 'r');
47 xlabel('Iteration'); ylabel('SSIM');
title('SSIM vs Iteration (Reg. Kaczmarz)'); grid on;

```

3.5 Part 3.5 – Regularized Kaczmarz with Different λ_{rel}

We define $\lambda = \sigma_1^2 \cdot \lambda_{\text{rel}}$ and try $\lambda_{\text{rel}} = [10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}]$. Since λ_{rel} is relative to σ_1^2 , we know the sensible range is $0 < \lambda_{\text{rel}} < 1$. Only the 10th iteration result is shown for each case.

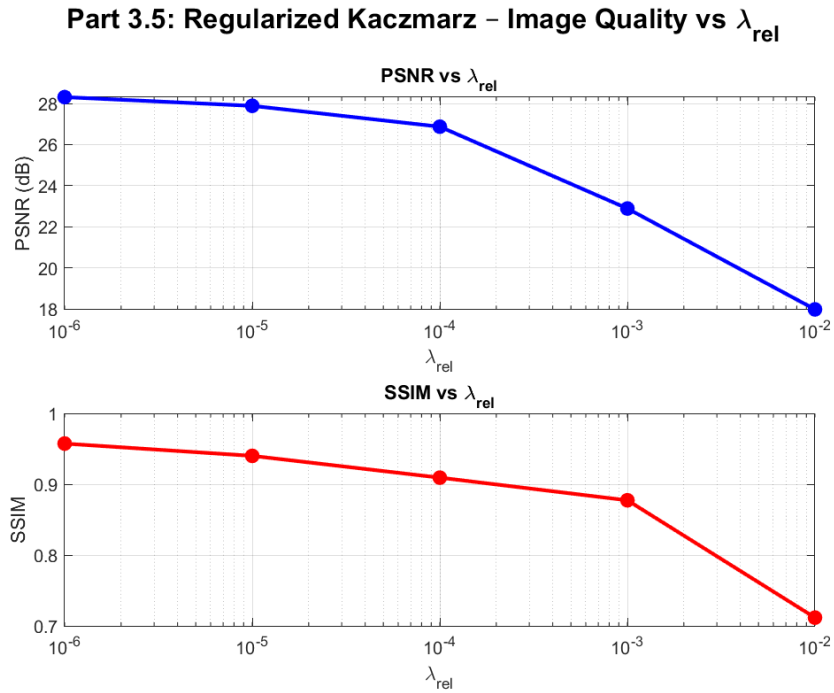


Figure 57: PSNR and SSIM as functions of λ_{rel} for regularized Kaczmarz (10th iteration).

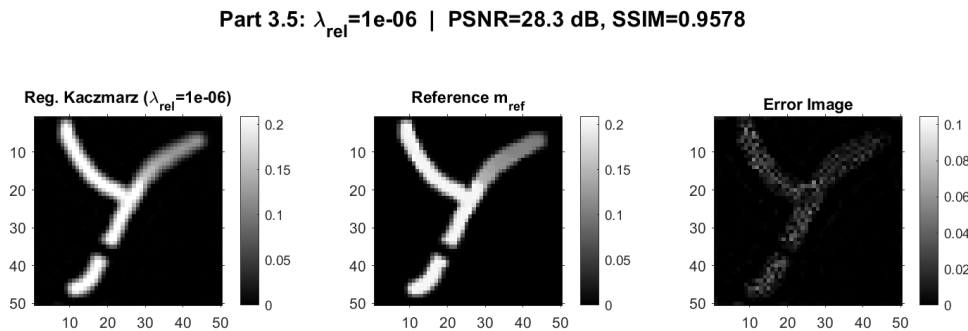


Figure 58: Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-6}$.

Part 3.5: $\lambda_{\text{rel}}=1\text{e-}05$ | PSNR=27.9 dB, SSIM=0.9405

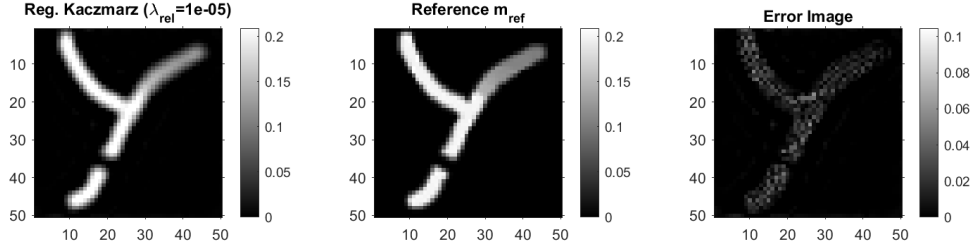


Figure 59: Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-5}$.

Part 3.5: $\lambda_{\text{rel}}=1\text{e-}04$ | PSNR=26.9 dB, SSIM=0.9097

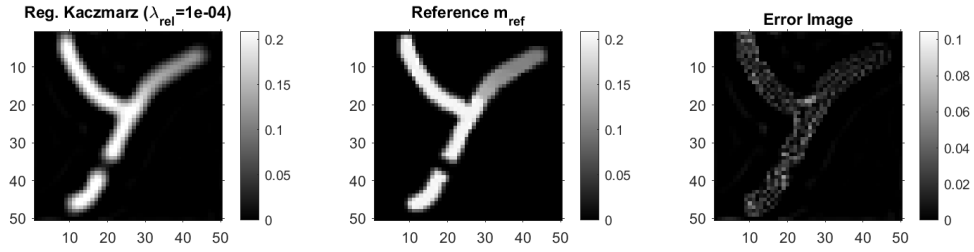


Figure 60: Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-4}$.

Part 3.5: $\lambda_{\text{rel}}=1\text{e-}03$ | PSNR=22.9 dB, SSIM=0.8777

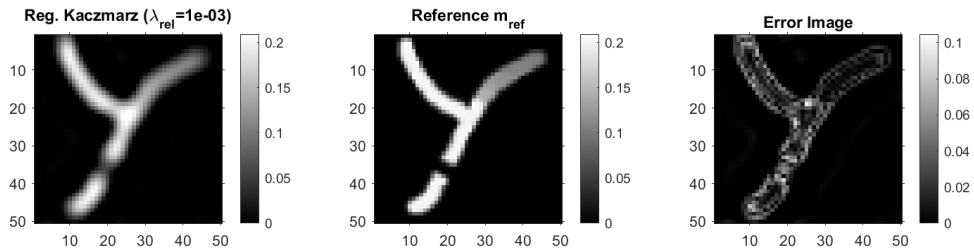


Figure 61: Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-3}$.

Part 3.5: $\lambda_{\text{rel}}=1\text{e-}02$ | PSNR=18.0 dB, SSIM=0.7118

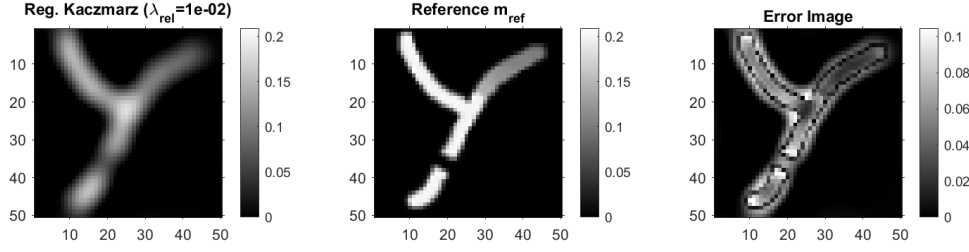


Figure 62: Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-2}$.

Comments: The best PSNR is 28.31 dB at $\lambda_{\text{rel}} = 10^{-6}$ (weakest regularization), with SSIM = 0.9578. As λ_{rel} increases, regularization becomes stronger: at 10^{-2} , PSNR drops to 17.98 dB. In this noiseless setting, weaker regularization performs better because there is no noise to suppress. The optimal λ_{rel} in the noiseless case is the smallest tested value (10^{-6}).

Code

```

1 lambda_rel_arr = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2];
2 N_lrel = length(lambda_rel_arr);
3 psnr_lrel = zeros(1, N_lrel);
4 ssim_lrel = zeros(1, N_lrel);
5
6 for t = 1:N_lrel
7     lam = sigma_vals(1)^2 * lambda_rel_arr(t);
8     c_t = zeros(n_cols_full, 1);
9
10    for iter = 1:10
11        order = randperm(n_rows_full);
12        for sub = 1:n_rows_full
13            idx = order(sub);
14            row_i = S(idx, :);
15            c_t = c_t + (u(idx) - row_i * c_t) / (row_i * row_i' +
16                lam) * row_i';
17        end
18        c_t(c_t < 0) = 0;
19    end
20
21    ima_t = reshape(c_t, 50, 50);
22    ima_t_norm = double(ima_t) / max_val;
23    psnr_lrel(t) = psnr(ima_t_norm, ref_norm);
24    ssim_lrel(t) = ssim(ima_t_norm, ref_norm);
25
26    err_t = abs(ima_t - phantom_ref);
27
28    figure; colormap(gray);

```

```

28     subplot(1,3,1); imshow(ima_t, [0 max_val]); colorbar;
29         title(sprintf('Reg. Kaczmarz (lrel=%.0e)',
30             lambda_rel_arr(t))); axis on;
31 subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
32         title('Reference m_{ref}'); axis on;
33 subplot(1,3,3); imshow(err_t, [0 0.5*max_val]); colorbar;
34         title('Error Image'); axis on;
35
36     fprintf(' lambda_rel = %.0e: PSNR = %.2f dB, SSIM = %.4f\n',
37         ...
38         lambda_rel_arr(t), psnr_lrel(t), ssim_lrel(t));
39
40 end
41
42 figure;
43 subplot(2,1,1);
44 semilogx(lambda_rel_arr, psnr_lrel, 'b-o', 'LineWidth', 2, '
45     MarkerFaceColor','b');
46 xlabel('\lambda_{rel}'); ylabel('PSNR (dB)');
47 title('PSNR vs \lambda_{rel}'); grid on;
48
49 subplot(2,1,2);
50 semilogx(lambda_rel_arr, ssim_lrel, 'r-o', 'LineWidth', 2, '
51     MarkerFaceColor','r');
52 xlabel('\lambda_{rel}'); ylabel('SSIM');
53 title('SSIM vs \lambda_{rel}'); grid on;

```

3.6 Part 3.6 – Effects of Measurement Noise (Standard Kaczmarz)

White Gaussian noise with zero mean and standard deviation $\max_norm/1000$ is added to the measurement vector u . The noise is *not* added to the system matrix. Part 3.3 is then repeated with the noisy u .

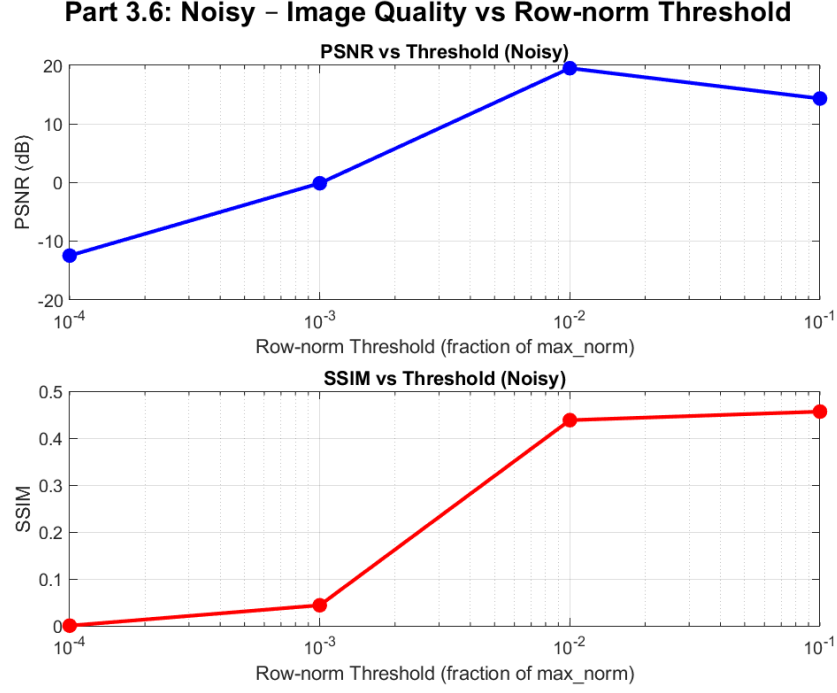


Figure 63: PSNR and SSIM vs. row-norm threshold for noisy standard Kaczmarz.

Part 3.6: noisy, thresh=1e-04 | PSNR=-12.5 dB, SSIM=0.0006

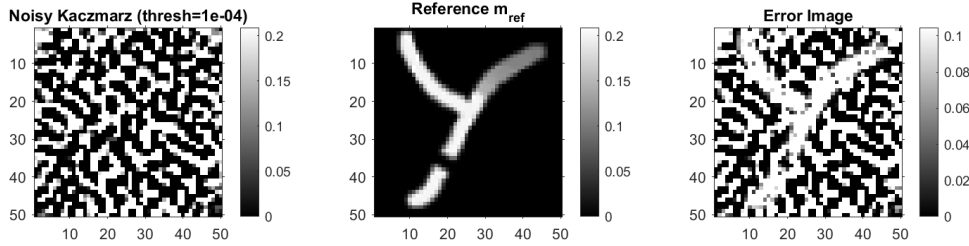


Figure 64: Noisy Standard Kaczmarz (iter 10): Threshold = 10^{-4} .

Part 3.6: noisy, thresh=1e-03 | PSNR=-0.1 dB, SSIM=0.0441

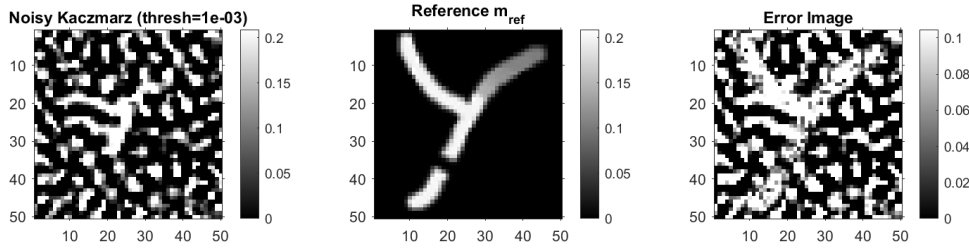


Figure 65: Noisy Standard Kaczmarz (iter 10): Threshold = 10^{-3} .

Part 3.6: noisy, thresh=1e-02 | PSNR=19.5 dB, SSIM=0.4390

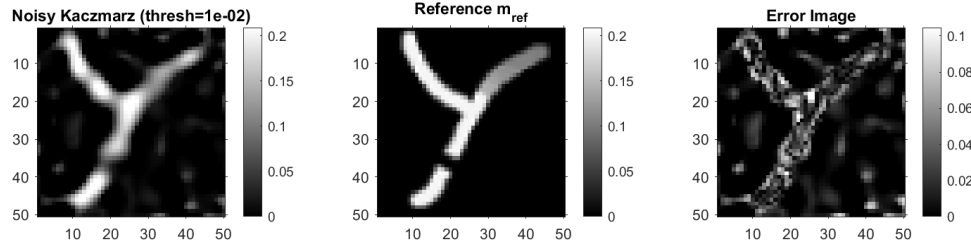


Figure 66: Noisy Standard Kaczmarz (iter 10): Threshold = 10^{-2} .

Part 3.6: noisy, thresh=1e-01 | PSNR=14.3 dB, SSIM=0.4573

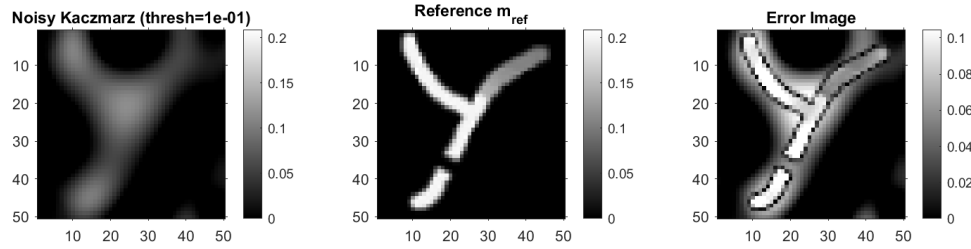


Figure 67: Noisy Standard Kaczmarz (iter 10): Threshold = 10^{-1} .

Comments: Noise dramatically changes the behavior. With threshold 10^{-4} , PSNR drops to -12.49 dB because many rows with small energy amplify the noise. The threshold 10^{-2} now gives the best result (PSNR = 19.50 dB, SSIM = 0.4390), shifting the optimal threshold significantly toward stronger thresholding compared to the noiseless case. This demonstrates that in the presence of noise, aggressive row-norm thresholding is essential.

Code

```

1 noise_std = max_norm / 1000;
2 u_noisy = u + noise_std * randn(size(u));
3
4 psnr_thresh_noisy = zeros(1, N_thresh);
5 ssim_thresh_noisy = zeros(1, N_thresh);
6
7 for t = 1:N_thresh
8     thresh_val = max_norm * thresh_factors(t);
9     keep = row_norms >= thresh_val;
10    S_t = S(keep, :);
11    u_t = u_noisy(keep);
12
13    c_t = zeros(n_cols, 1);

```

```

14
15     for iter = 1:10
16         order = randperm(size(S_t, 1));
17         for sub = 1:size(S_t, 1)
18             idx = order(sub);
19             row_i = S_t(idx, :);
20             c_t = c_t + (u_t(idx) - row_i * c_t) / (row_i * row_i')
                * row_i';
21         end
22         c_t(c_t < 0) = 0;
23     end
24
25     ima_t = reshape(c_t, 50, 50);
26     ima_t_norm = double(ima_t) / max_val;
27     psnr_thresh_noisy(t) = psnr(ima_t_norm, ref_norm);
28     ssim_thresh_noisy(t) = ssim(ima_t_norm, ref_norm);
29
30     err_t = abs(ima_t - phantom_ref);
31
32     figure; colormap(gray);
33     subplot(1,3,1); imshow(ima_t, [0 max_val]); colorbar;
34     title(sprintf('Noisy Kaczmarz (thresh=%.0e)',
35                 thresh_factors(t))); axis on;
36     subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
37     title('Reference m_{ref}'); axis on;
38     subplot(1,3,3); imshow(err_t, [0 0.5*max_val]); colorbar;
39     title('Error Image'); axis on;
40
41     fprintf(' Threshold %.0e (noisy): PSNR = %.2f dB, SSIM = %.4f\n', ...
42         thresh_factors(t), psnr_thresh_noisy(t), ssim_thresh_noisy(t));
43
44 end
45
46 figure;
47 subplot(2,1,1);
48 semilogx(thresh_factors, psnr_thresh_noisy, 'b-o', 'LineWidth', 2,
49         'MarkerFaceColor', 'b');
50 xlabel('Threshold (max\_norm orani)'); ylabel('PSNR (dB)');
51 title('PSNR vs Threshold (Noisy)'); grid on;
52
53 subplot(2,1,2);
54 semilogx(thresh_factors, ssim_thresh_noisy, 'r-o', 'LineWidth', 2,
55         'MarkerFaceColor', 'r');
56 xlabel('Threshold (max\_norm orani)'); ylabel('SSIM');
57 title('SSIM vs Threshold (Noisy)'); grid on;

```

3.7 Part 3.7 – Effects of Noise (Regularized Kaczmarz)

Part 3.5 is repeated using the same noisy measurement vector u .

Part 3.7: Noisy – Image Quality vs λ_{rel}

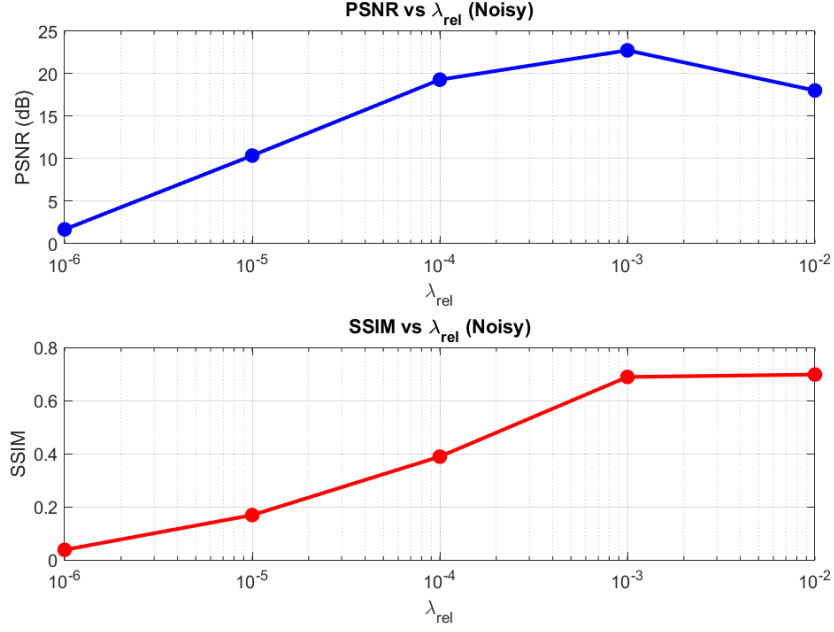


Figure 68: PSNR and SSIM vs. λ_{rel} for noisy regularized Kaczmarz.

Part 3.7: noisy $\lambda_{\text{rel}}=1\text{e-}06$ | PSNR=1.6 dB, SSIM=0.0383

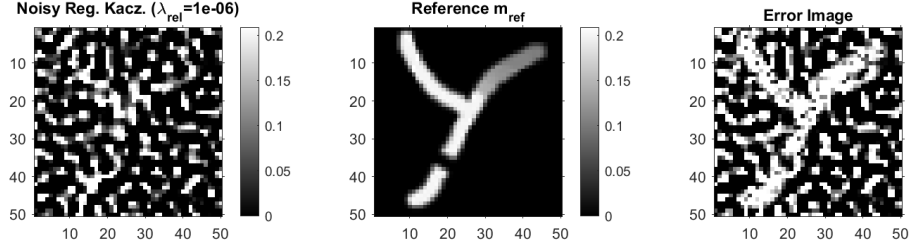


Figure 69: Noisy Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-6}$.

Part 3.7: noisy $\lambda_{\text{rel}}=1\text{e-}05$ | PSNR=10.3 dB, SSIM=0.1691

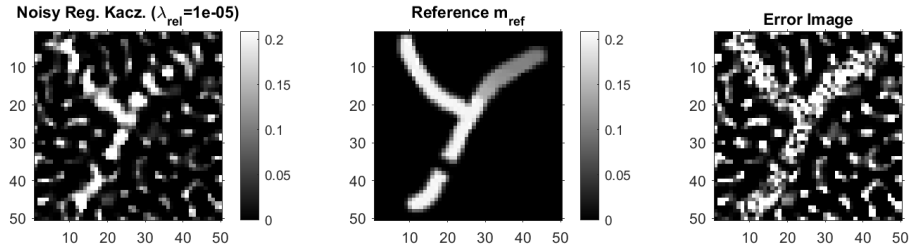


Figure 70: Noisy Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-5}$.

Part 3.7: noisy $\lambda_{\text{rel}}=1\text{e-}04$ | PSNR=19.3 dB, SSIM=0.3896

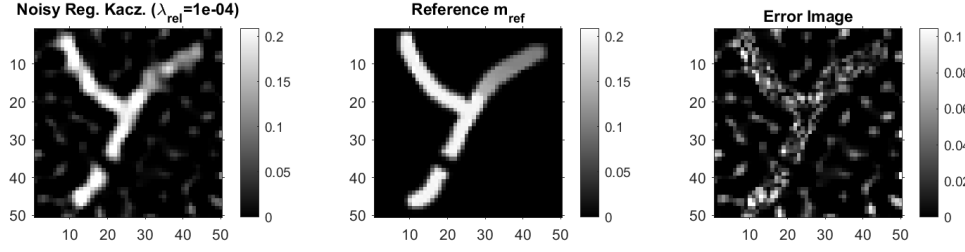


Figure 71: Noisy Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-4}$.

Part 3.7: noisy $\lambda_{\text{rel}}=1\text{e-}03$ | PSNR=22.7 dB, SSIM=0.6899

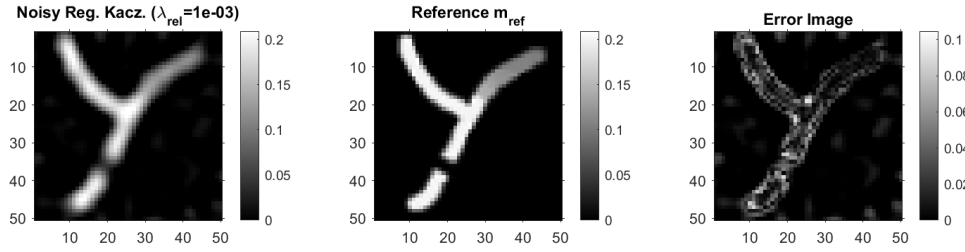


Figure 72: Noisy Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-3}$.

Part 3.7: noisy $\lambda_{\text{rel}}=1\text{e-}02$ | PSNR=18.0 dB, SSIM=0.6989

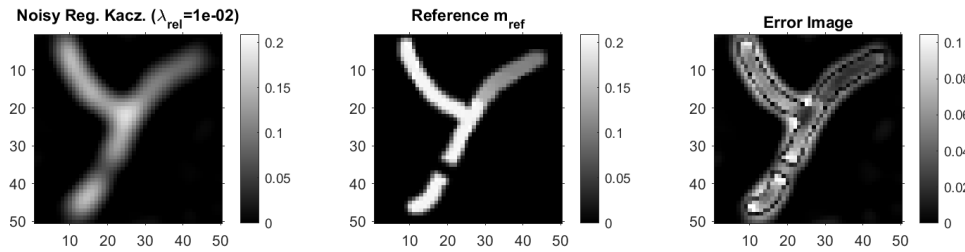


Figure 73: Noisy Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-2}$.

Comments: In the presence of noise, the optimal λ_{rel} shifts dramatically. At $\lambda_{\text{rel}} = 10^{-6}$, PSNR is only 1.64 dB (completely corrupted by noise), while the best result is at $\lambda_{\text{rel}} = 10^{-3}$ with PSNR = 22.74 dB and SSIM = 0.6899. This is a significant shift from the noiseless case where $\lambda_{\text{rel}} = 10^{-6}$ was optimal. Stronger regularization is needed to suppress the noise amplification. The regularized Kaczmarz handles noise more gracefully

than standard Kaczmarz with thresholding, since the regularization is applied uniformly to all rows.

Code

```

1 psnr_lrel_noisy = zeros(1, N_lrel);
2 ssim_lrel_noisy = zeros(1, N_lrel);
3
4 for t = 1:N_lrel
5     lam = sigma_vals(1)^2 * lambda_rel_arr(t);
6     c_t = zeros(n_cols_full, 1);
7
8     for iter = 1:10
9         order = randperm(n_rows_full);
10        for sub = 1:n_rows_full
11            idx = order(sub);
12            row_i = S(idx, :);
13            c_t = c_t + (u_noisy(idx) - row_i * c_t) / (row_i *
                row_i' + lam) * row_i';
14        end
15        c_t(c_t < 0) = 0;
16    end
17
18    ima_t = reshape(c_t, 50, 50);
19    ima_t_norm = double(ima_t) / max_val;
20    psnr_lrel_noisy(t) = psnr(ima_t_norm, ref_norm);
21    ssim_lrel_noisy(t) = ssim(ima_t_norm, ref_norm);
22
23    err_t = abs(ima_t - phantom_ref);
24
25    figure; colormap(gray);
26    subplot(1,3,1); imshow(ima_t, [0 max_val]); colorbar;
27    title(sprintf('Noisy Reg. Kacz. (lrel=%.0e)',
        lambda_rel_arr(t))); axis on;
28    subplot(1,3,2); imshow(phantom_ref, [0 max_val]); colorbar;
29    title('Reference m_{ref}'); axis on;
30    subplot(1,3,3); imshow(err_t, [0 0.5*max_val]); colorbar;
31    title('Error Image'); axis on;
32
33    fprintf(' lambda_rel = %.0e (noisy): PSNR = %.2f dB, SSIM =
        %.4f\n', ...
        lambda_rel_arr(t), psnr_lrel_noisy(t), ssim_lrel_noisy(t));
34
35 end
36
37
38 figure;
39 subplot(2,1,1);
40 semilogx(lambda_rel_arr, psnr_lrel_noisy, 'b-o', 'LineWidth', 2, '
    MarkerFaceColor','b');
41 xlabel('\lambda_{rel}'); ylabel('PSNR (dB)');
42 title('PSNR vs \lambda_{rel} (Noisy)'); grid on;
43

```

```
44 subplot(2,1,2);
45 semilogx(lambda_rel_arr, ssim_lrel_noisy, 'r-o', 'LineWidth', 2, '
    MarkerFaceColor','r');
46 xlabel('\lambda_{rel}'); ylabel('SSIM');
47 title('SSIM vs \lambda_{rel} (Noisy)'); grid on;
```

4 Part IV – Open MPI Data

4.1 Part 4.1 – Loading and Viewing Open MPI Data

The file `su_openmpi.mat` contains experimental MPI data from the Open MPI database. The system matrix S has size 3056×50653 and the measurement vector u has size 3056×1 . This data covers a 3D Lissajous trajectory with $37 \times 37 \times 37$ grid size and signal from two receive coils. Three columns of the system matrix are plotted.

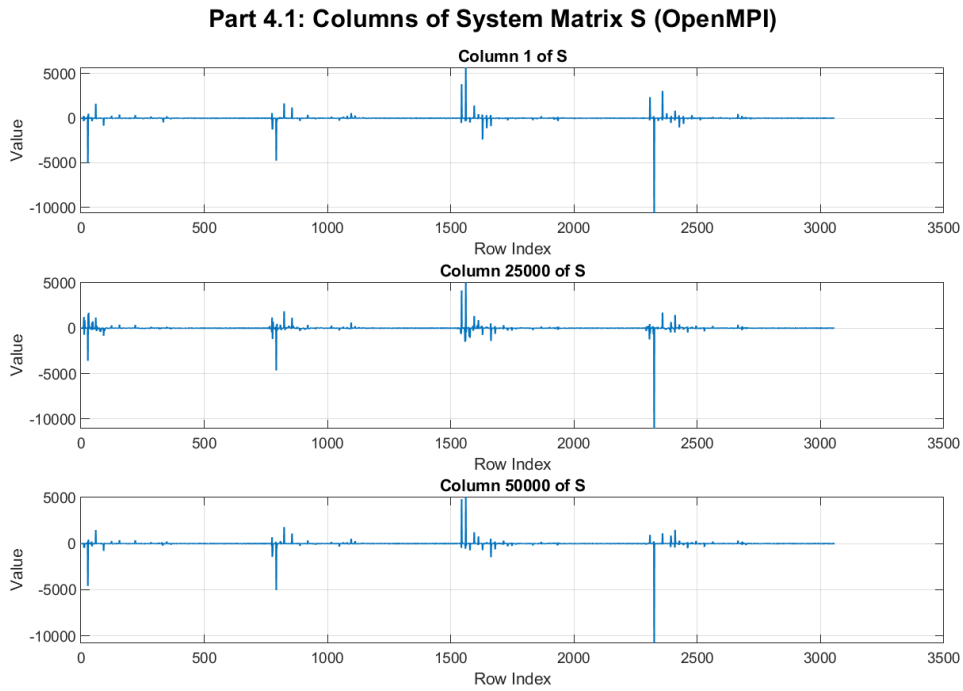


Figure 74: Three columns of the OpenMPI system matrix S (positions 1, 25000, 50000).

Code

```
1 fprintf('\nPart 4.1: Loading su_openmpi.mat (may take a while)... \n\n');
2 load('su_openmpi.mat');
3
4 fprintf('Size of S: %d x %d\n', size(S,1), size(S,2));
5 fprintf('Size of u: %d x %d\n', size(u,1), size(u,2));
6
7 locs_4 = [1, 25000, 50000];
8 figure;
9 for i = 1:3
10     subplot(3,1,i);
11     plot(S(:, locs_4(i)), 'LineWidth', 1);
12     xlabel('Row Index');
13     ylabel('Value');
14     title(['Column ' num2str(locs_4(i)) ' of S']);
15     grid on;
16 end
```

4.2 Part 4.2 – SVD of OpenMPI System Matrix

The compact SVD of the OpenMPI system matrix is computed. Due to the large size of S , this computation may take several minutes.

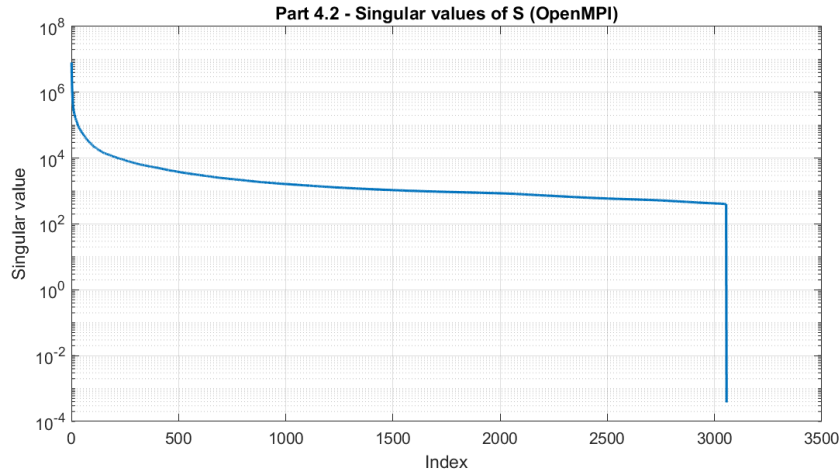


Figure 75: Singular values of the OpenMPI system matrix (log scale).

The condition number is $\text{Cond}(S) = 2.13 \times 10^{10}$, which is large but significantly smaller than the simulated system matrix in Part I.

Code

```
1 fprintf('\nPart 4.2: Computing compact SVD of S (OpenMPI, this will
   take a while)... \n');
2 [U4, Sigma4, V4] = svd(S, 'econ');
3
4 sigma_vals_4 = diag(Sigma4);
5
6
7 figure;
8 semilogy(sigma_vals_4, 'LineWidth', 1.5);
9 xlabel('Index');
10 ylabel('Singular value');
11 title('Part 4.2 - Singular values of S (OpenMPI)');
12 grid on;
13
14 cond_4 = sigma_vals_4(1) / sigma_vals_4(end);
15 fprintf('Condition number (OpenMPI): %.4e\n', cond_4);
```

4.3 Part 4.3 – Regularized Kaczmarz on OpenMPI Data

The regularized Kaczmarz method is applied to the OpenMPI data with $\lambda = \sigma_1 \sigma_N$ and no row-norm thresholding. After 10 iterations, the reconstructed vector c is reshaped into a 3D image volume of size $37 \times 37 \times 37$. All 37 slices are displayed as a montage.

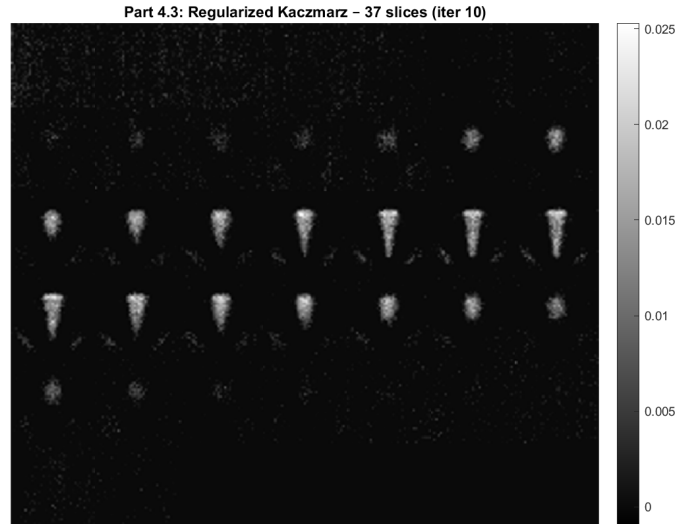


Figure 76: All 37 slices of the 3D MPI reconstruction using regularized Kaczmarz ($\lambda = \sigma_1 \sigma_N$, iteration 10).

Code

```

1 lambda_43 = sigma_vals_4(1) * sigma_vals_4(end);
2 fprintf('\nPart 4.3: Regularized Kaczmarz (lambda = sigma_1*sigma_N
   = %.4e)\n', lambda_43);
3
4 [n_rows_4, n_cols_4] = size(S);
5 c_4 = zeros(n_cols_4, 1);
6
7 for iter = 1:10
8     order = randperm(n_rows_4);
9     for sub = 1:n_rows_4
10         idx = order(sub);
11         row_i = S(idx, :);
12         c_4 = c_4 + (u(idx) - row_i * c_4) / (row_i * row_i' +
           lambda_43) * row_i';
13     end
14     c_4(c_4 < 0) = 0;
15     fprintf(' Iter %d done.\n', iter);
16 end
17
18 ima_4 = reshape(c_4, 37, 37, 37);
19
20 figure;
21 montage(reshape(ima_4, [37, 37, 1, 37]), 'DisplayRange', []);
22 title('Part 4.3: Regularized Kaczmarz - 37 slices (iter 10)');
23 colormap(gray); colorbar;

```

4.4 Part 4.4 – Regularized Kaczmarz for Different λ_{rel} (Open-MPI)

The regularized Kaczmarz is repeated with $\lambda = \sigma_1^2 \cdot \lambda_{\text{rel}}$ for $\lambda_{\text{rel}} = [10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}]$. The 10th iteration montage is displayed for each case.

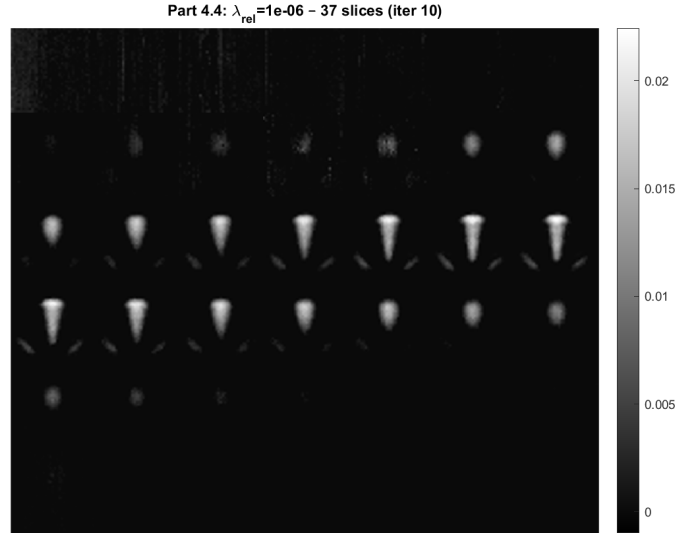


Figure 77: OpenMPI Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-6}$.

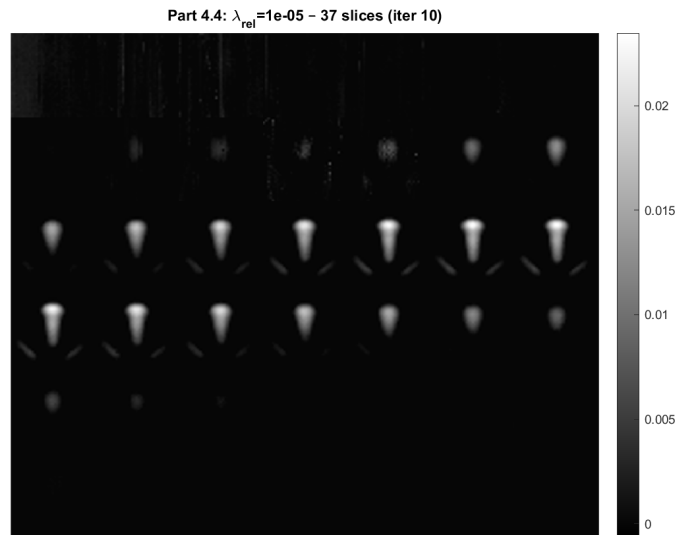


Figure 78: OpenMPI Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-5}$.

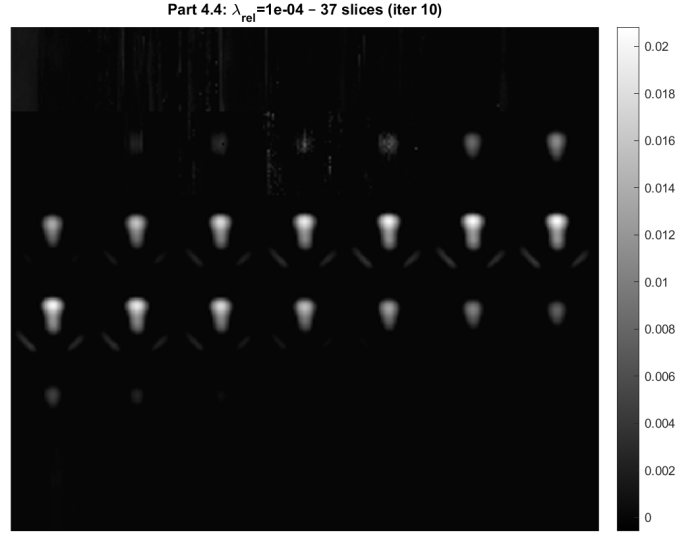


Figure 79: OpenMPI Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-4}$.

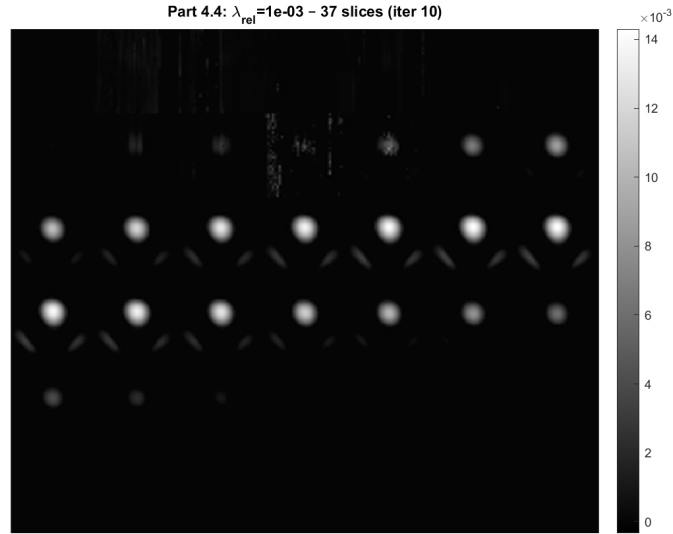


Figure 80: OpenMPI Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-3}$.

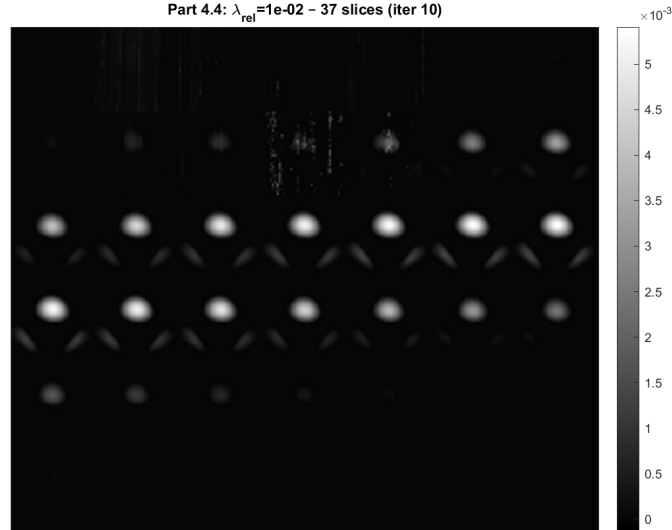


Figure 81: OpenMPI Regularized Kaczmarz (iter 10): $\lambda_{\text{rel}} = 10^{-2}$.

Comments: Without a ground-truth reference, image quality is assessed visually. Very small λ_{rel} values (e.g., 10^{-6}) produce noisy images with many artifacts. Very large λ_{rel} values (e.g., 10^{-2}) over-smooth the image and reduce contrast. Based on visual inspection of the montages, an intermediate value such as $\lambda_{\text{rel}} \approx 10^{-4}$ or 10^{-3} provides the best trade-off between noise and resolution, where the phantom features are clearly visible with good contrast and minimal noise.

Code

```

1 lambda_rel_4 = [1e-6, 1e-5, 1e-4, 1e-3, 1e-2];
2 N_lrel4 = length(lambda_rel_4);
3
4 for t = 1:N_lrel4
5     lam = sigma_vals_4(1)^2 * lambda_rel_4(t);
6     c_t = zeros(n_cols_4, 1);
7
8     for iter = 1:10
9         order = randperm(n_rows_4);
10        for sub = 1:n_rows_4
11            idx = order(sub);
12            row_i = S(idx, :);
13            c_t = c_t + (u(idx) - row_i * c_t) / (row_i * row_i' +
14                lam) * row_i';
15        end
16        c_t(c_t < 0) = 0;
17    end
18
19    ima_t = reshape(c_t, 37, 37, 37);
20
21    figure;
22    montage(reshape(ima_t, [37, 37, 1, 37]), 'DisplayRange', []);

```

```
22     title(sprintf('Part 4.4: lambda_{rel}=%.0e - 37 slices (iter  
23         10)', lambda_rel_4(t)));  
24     colormap(gray); colorbar;  
25  
25     fprintf(' lambda_rel = %.0e done.\n', lambda_rel_4(t));  
26 end
```